Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Reliability & Risk Engineering

Simon Scandella

Fast evaluation of the status of batteries using physical simulations and simulation based statistical inference with normalizing flows

Master Thesis

RRE – Reliability and Risk Engineering Lab
Swiss Federal Institute of Technology (ETH) Zurich

Expert: Prof. Dr. Giovanni Sansavini
Supervisors: Dr. Anna Varbella - RRE; Dr. Kai Hencken - ABB Corporate Research

Zürich, March 28, 2026

Abstract

Accurate estimation of a battery's internal states, in particular the State of Charge (SoC) and State of Health (SoH), is essential for the safe and efficient operation of batteries. However, the inverse mapping from measured current-voltage data to internal states is inherently ill-posed. Different combinations of SoC, capacity, internal parameters and measurement noise can produce nearly indistinguishable terminal voltage trajectories. These characteristics motivate probabilistic approaches that explicitly represent uncertainty instead of relying solely on deterministic point estimates. This thesis investigates the application of simulation-based Bayesian inference to dynamic battery state estimation. An equivalent-circuit model (ECM) serves as a forward simulator to generate realistic current-voltage trajectories under varying operating conditions. A neural posterior estimator based on conditional normalizing flows is trained to learn the inverse mapping from measurements to posterior distributions over SoC and SoH. The amortized nature of the approach enables efficient inference, as posterior samples can be obtained without repeated simulator evaluations once training is completed. The results demonstrate accurate, well-calibrated SoC inference across operating regimes, while SoH identifiability depends strongly on sufficient cumulative charge throughput. Models trained exclusively on synthetic ECM simulations generalize successfully to experimental battery data for SoC inference. Robustness analyses indicate that exact replication of real current statistics is not required, provided that the training data include sufficiently rich dynamic and rest phases, whereas systematic mismatch leads to physically interpretable performance degradation. Compared to a traditional likelihood-free method (Sequential Monte Carlo Approximate Bayesian Computation), the proposed method achieves speedups exceeding two orders of magnitude while maintaining comparable posterior quality. A sequential inference extension was developed to propagate posterior information across consecutive measurement windows, but did not yield consistent improvements for SoC estimation. The results suggest that sequential methods may be more relevant for weakly identifiable quantities such as SoH. Overall, the findings show that amortized neural posterior estimation provides a computationally efficient, physically consistent framework for probabilistic battery-state estimation.

Acknowledgements

I would like to express my sincere gratitude to my academic supervisor, Dr. Anna Varbella, for her guidance, valuable feedback, and regular discussions throughout this thesis. I am especially grateful to Dr. Kai Hencken at ABB Schweiz AG for his continuous support, insightful discussions, and strong engagement during the project. His guidance and the opportunity to carry out this work at ABB were essential to the successful completion of this thesis. I also thank ABB Schweiz AG for providing the opportunity to conduct this thesis in an industrial research environment. Furthermore, I would like to thank Lorenzo Tiberi and Matthias Baur for providing access to battery data and reference results, which were essential for the experimental evaluation in this work. I also acknowledge Prof. Dr. Giovanni Sansavini for giving me the opportunity to carry out this thesis within his group. Finally, I would like to thank my parents, Ilda and Corsin Scandella, for their continuous support and encouragement throughout my studies. Without them, this work would not have been possible.

Contents

List of Acronyms

SoC	State of Charge
SoH	State of Health - fraction of remaining capacity
SMC-ABC	Sequential Monte Carlo Approximate Bayesian Computation
ECM	Equivalent Circuit Model
DEKF	Dual Extended Kalman Filter
SBI	Simulation-based Inference
NPE	Neural Posterior Estimation
sw-NPE	short window Neural Posterior Estimation
ad-NPE	all data Neural Posterior Estimation
seq-NPE	sequential Neural Posterior Estimation (distinct from SNPE)
NF	Normalizing Flow
NN	Neural Network
INN	Invertible Neural Network
PPC	Posterior Predictive Check
SBC	Simulation-based Calibration
OCV	Open Circuit Voltage
RC	Resistor–Capacitor (network element)
KLE	Karhunen–Loève Expansion
MCMC	Markov Chain Monte Carlo
MAD	Median Absolute Deviation
VI	Variational Inference
PDF	Probability Density Function
CDF	Cumulative Distribution Function
ECDF	Empirical Cumulative Distribution Function
RMSE	root mean squared error
GPU	Graphics Processing Unit
HPC	High-Performance Computing

List of Figures

List of Tables

Chapter 1

Introduction

Accurate knowledge of a battery's internal states, such as the State of Charge (SoC), representing the remaining energy stored in the battery, and the State of Health (SoH), describing the degradation of the battery capacity relative to its nominal capacity, is essential for the safe and efficient operation of electric vehicles and stationary energy storage systems. Reliable estimates of these hidden states enable improved energy-management strategies and accurate range prediction. However, these quantities cannot be measured directly during operation and must instead be inferred from observable signals such as voltage and current measurements. Estimating battery states from current-voltage data therefore enables online monitoring of the battery during operation. However, inferring these internal states from measurements is challenging because the terminal voltage does not directly reveal the underlying battery states. To understand where the information about these hidden states is encoded in the measurements, it is useful to consider the equivalent circuit model (ECM) framework commonly used for battery state estimation. In this representation, the terminal voltage is expressed as the sum of the open-circuit voltage (OCV), the voltage drop across the internal resistance, and the voltages across RC elements that account for transient electrochemical effects. Among these contributions, the OCV is the primary carrier of information about the State of Charge, as it is a nonlinear function of SoC. The evolution of SoC, however, depends on the effective battery capacity and therefore on the State of Health. As batteries age, their usable capacity decreases. Consequently, the same current drawn from two batteries with different capacities will change their SoC at different rates: a battery with reduced capacity will reach lower SoC levels sooner when subjected to the same discharge current. Because the OCV depends on SoC, this difference in SoC evolution leads to diverging voltage trajectories over time. In this way, the State of Health influences the measured voltage signal indirectly through its effect on the SoC dynamics. During dynamic operation, however, the voltage contributions induced by load current, namely the voltage drop across the internal resistance and the transient RC voltages, obscure the OCV component. As a result, different combinations of SoC, capacity, and internal parameters can produce nearly identical terminal voltage trajectories, rendering the inverse mapping from measurements to internal states ill-posed. The most informative operating condi-

tion for SoC estimation is therefore a sufficiently long rest period with negligible current flow. Under such conditions, the voltage drop across the internal resistance vanishes and the RC elements relax, so that the terminal voltage approaches the OCV. The measured voltage can then be mapped directly to the corresponding state of charge. Even under such favorable conditions, however, uncertainty remains unavoidable. Measurement noise and sensor calibration errors introduce ambiguity in the inferred OCV value, which translates into a range of feasible SoC values rather than a unique estimate. This effect is particularly pronounced for batteries with flat OCV–SoC characteristics, where small voltage deviations correspond to large variations in SoC. In addition to the limitations of voltage-based estimation discussed above, current measurements must also be taken into account. While voltage provides an absolute reference through the OCV–SoC relationship, current measurements can be integrated over time (Coulomb counting) to estimate changes in SoC. However, Coulomb counting is sensitive to measurement noise and sensor bias, leading to error accumulation and drift over time. Accurate state estimation therefore requires combining both sources of information. Consequently, battery state estimation is inherently uncertain, motivating probabilistic estimation approaches that explicitly represent and propagate uncertainty instead of collapsing it into a single point estimate.

Among probabilistic approaches, two main families can be distinguished. The first are **likelihood-based methods**, which rely on an explicit mathematical model that links the hidden states or parameters to the measured quantities. These methods use this likelihood function to update prior beliefs about the internal states through techniques such as Bayesian filtering, variational inference, or Markov Chain Monte Carlo sampling. A representative example is provided by [3], who applied Bayesian parameter inference to the single-particle model with electrolyte dynamics (SPMe). In their work, explicit prior and likelihood models were defined for the measured voltage data, and posterior distributions over electrochemical parameters were approximated using Markov Chain Monte Carlo (MCMC) sampling. Such likelihood-based Bayesian approaches provide well-calibrated uncertainty estimates but require an explicit mathematical formulation of the likelihood, which is not always available or straightforward to specify for complex battery models.

The second family comprises **likelihood-free methods**, most prominently those falling under the umbrella of *simulation-based inference* (SBI). The key characteristic of this class of methods is that they do not require an explicit analytical expression for the likelihood function, which may be unavailable or intractable. Instead of evaluating $p(x \mid \theta)$ directly, one only requires a stochastic *forward simulator* capable of generating synthetic data samples $x \sim p(x \mid \theta)$ for given parameters θ drawn from the prior. This replacement of the explicit likelihood evaluation with a generative simulator defines the simulation-based inference paradigm, encompassing both classical and neural approaches. Early likelihood-free methods such as Approximate Bayesian Computation (ABC) and Bayesian Synthetic Likelihood (BSL) approximate the posterior by comparing synthetic and observed data through summary statistics or discrepancy measures [4, 5].

More recently, **neural density estimators** have been proposed as an alternative to traditional likelihood-free methods, often in an amortized setting. Here, amortized refers to the fact that the computational cost associated with repeated simulator evaluations during inference is shifted to an upfront training phase, allowing the trained model to perform fast inference for new observations. A commonly used approach within this class of methods is neural posterior estimation (NPE), which directly learns an approximation of the posterior distribution. Unlike methods such as ABC or BSL, which require repeated simulator evaluations during inference, NPE requires only a single forward pass through the neural networks to obtain one posterior sample. Normalizing-flow-based neural posterior estimators have also recently been applied to battery modeling. For example, [6] trained a normalizing-flow-based NPE on simulated capacity-over-cycles data to estimate parameters of a stochastic battery degradation model. Their work focuses on identifying degradation dynamics from capacity trajectories rather than estimating operational states from current-voltage measurements. Although these results demonstrate the potential of neural density estimators for battery modeling, their application to probabilistic battery state estimation from operational measurements remains largely unexplored. In particular, the use of neural posterior estimation for inferring operational states such as SoC and SoH from time-dependent current-voltage data during realistic battery operation has received comparatively little attention. Furthermore, existing neural posterior estimation approaches typically treat observations independently during inference and do not explicitly propagate posterior information between consecutive measurement segments. Building on these observations, this thesis investigates neural posterior estimation as a probabilistic framework for dynamic battery-state estimation from current-voltage measurements under realistic load profiles. In addition, a sequential inference strategy is explored in which posterior information inferred from earlier data segments is propagated forward to subsequent observations.

In this work, the proposed approach is implemented using the *BayesFlow* [7] framework, an open-source Python package designed for rapid prototyping of simulation-based inference methods. BayesFlow provides a modular interface for constructing simulators, priors, and inference networks, as well as built-in diagnostics for simulation-based calibration (SBC). Normalizing flows are particularly well suited for this task because they construct expressive posterior distributions through invertible transformations of a simple base density, allowing flexible modeling of non-Gaussian uncertainty while enabling efficient sampling within an amortized inference framework. The main contributions of this thesis are summarized as follows:

- Application of neural posterior estimation to probabilistic battery state estimation from time-dependent current-voltage measurements.
- Demonstration that a neural posterior estimator trained exclusively on simulated equivalent-circuit-model trajectories can generalize to real experimental battery data for state-of-charge estimation.
- Investigation of a sequential inference strategy that propagates posterior information between consecutive measurement segments.

Chapter 2

Methods

2.1 Simulator

To perform simulation-based inference (SBI), a simulator capable of generating synthetic data is required. In principle, any battery simulator could be used. In this thesis, a second-order Thevènin equivalent circuit model (ECM) is adopted (Figure 2.1) due to its simplicity, relatively low computational cost at simulation time, and ease of implementation. In addition, a three-RC variant of the ECM is employed in later stages of the work.

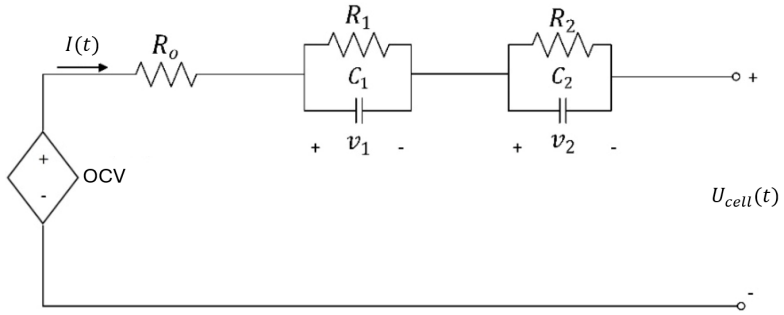


Figure 2.1: Two-RC Thevènin equivalent circuit model used in this work (adapted from [1]). Here, OCV denotes the open-circuit voltage, $U_{cell}(t)$ the terminal voltage, R_i and C_i the resistances and capacitances of the RC branches, and v_i the corresponding branch voltages. The three-RC variant used later in this work is obtained by extending this circuit with an additional RC branch of identical structure.

The battery parameters, including the resistances (R_i), capacitances (C_i), the OCV curve and the nominal capacity, are taken from literature [1] for an NMC lithium-ion cell. Crucially, these parameters are functions of the state of charge.

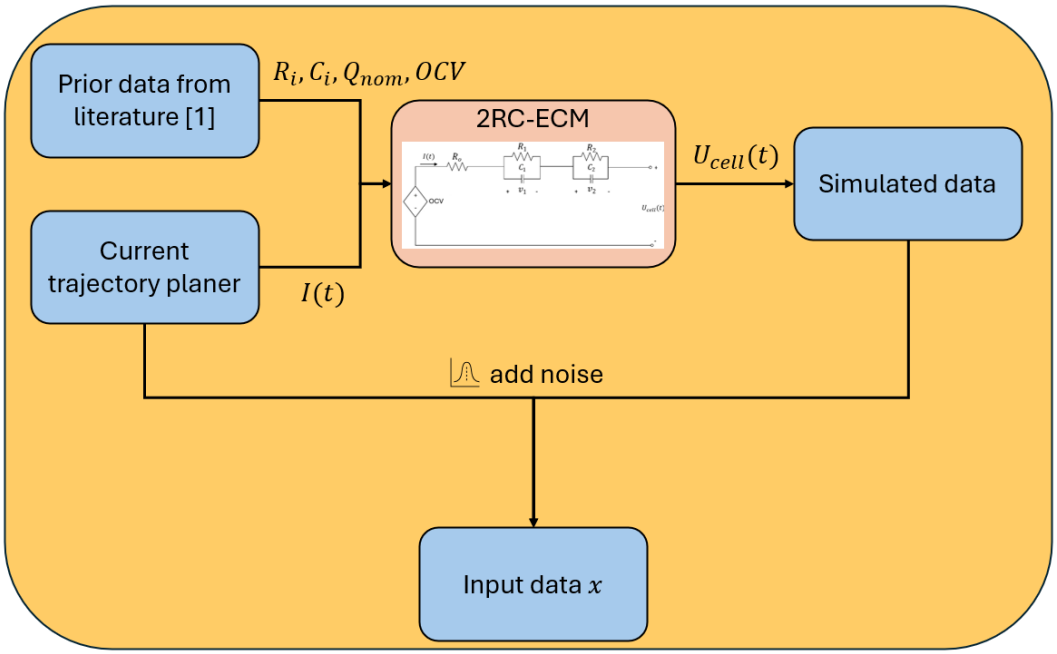


Figure 2.2: Overview of the battery simulation workflow. Current trajectories and cell parameters are passed to an ECM, which computes the terminal voltage response. Measurement noise and bias terms are subsequently applied. The resulting signals, together with a normalized time axis, are concatenated to form the model input tensor $\mathbf{x}$. The circuit illustration within the diagram is adapted from the same source as Figure 2.1 [1].

Battery parameters (R_i, C_i, OCV, and the nominal capacity Q_{nom}) are combined with a planned current trajectory $I(t)$ (described in Section 2.2) and passed to the second-order Thevènin equivalent circuit model (ECM). The ECM produces a terminal voltage trace $U_{\text{cell}}(t)$. To emulate measurement uncertainty and modeling imperfections, both signal-level perturbations and parameter-level variations are introduced, as illustrated in Figure 2.2. Specifically, additive noise and bias terms are applied to the simulated current and voltage signals, while the underlying ECM parameters (including R_i, C_i, and the OCV curve) are subject to stochastic variability and bias during data generation. The resulting signals, together with a normalized time axis, are then combined to form the model input tensor $\mathbf{x}$. Details on the noise model, parameter jitter, and signal scaling are provided in Section 2.1.1 and Appendix A.

Equations (2.1)-(2.2) describe the continuous-time dynamics of the 2RC Thevenin model, while Equation (2.3) defines the Coulomb-counting relation used to compute the state of charge (SoC). The ECM parameters R_i, C_i, and OCV are modeled as functions SoC and are therefore evaluated based on the current SoC during simulation. In practice, the simulator operates in discrete time using sampled current measurements, and the ECM dynamics are implemented using the discrete-time formulations given in Equations (2.4)-(2.7).

Continuous-time model.

$$U_{\text{cell}}(t) = \text{OCV}(\text{SoC}(t)) - R_0(\text{SoC}(t))\, I(t) - \sum_{i=1}^{2} v_i(t), \tag{2.1}$$

$$\dot{v}_i(t) = -\frac{1}{\tau_i(\text{SoC}(t))}\, v_i(t) + \frac{R_i(\text{SoC}(t))}{\tau_i(\text{SoC}(t))}\, I(t), \qquad i = 1, 2, \tag{2.2}$$

$$\text{SoC}(t) = \text{SoC}(t_0) - \frac{1}{\text{SoH} \cdot Q_{\text{NOM}}} \int_{t_0}^{t} I(s)\, ds. \tag{2.3}$$

with time constants

$$\tau_i(\text{SoC}(t)) = R_i(\text{SoC}(t))\, C_i(\text{SoC}(t)), \qquad i = 1, 2.$$

Discrete-time model.

$$U_{\text{cell}}[k] = \text{OCV}(\text{SoC}[k]) - R_0(\text{SoC}[k])\, I[k] - \sum_{i=1}^{2} v_i[k], \tag{2.4}$$

$$v_i[k] = a_i[k]\, v_i[k-1] + (1 - a_i[k])\, R_i[k]\, I[k], \qquad i = 1, 2, \tag{2.5}$$

$$a_i[k] = \exp\!\left(-\frac{\Delta t}{\tau_i[k]}\right), \tag{2.6}$$

$$\text{SoC}[k] = \text{SoC}[k-1] - \frac{\Delta t}{\text{SoH} \cdot Q_{\text{NOM}}} I[k], \qquad \text{SoC}[0] \text{ given.} \tag{2.7}$$

where the parameters $R_i[k] = R_i(\text{SoC}[k])$, $C_i[k] = C_i(\text{SoC}[k])$, and $\tau_i[k] = R_i[k]\, C_i[k]$ are evaluated as functions of the current SoC. The state of health (SoH) is defined as

$$\text{SoH} = \frac{Q_{\text{eff}}}{Q_{\text{NOM}}},$$

where Q_{eff} denotes the effective capacity and Q_{NOM} the nominal capacity, both assumed to be expressed in ampere-seconds (As). This formulation highlights that SoH directly influences the SoC dynamics through the effective capacity. For a 3RC Thevènin model, the formulation extends straightforwardly to $i = 1, 2, 3$.

2.1.1 Implementation Details

In order to reflect realistic uncertainty in synthetic data generation, two distinct sources of variability are modeled:

1. **Parameter variability**, representing cell-to-cell differences and uncertainty in the equivalent circuit model parameters (e.g. OCV, R_0, R_i, C_i).
2. **Measurement noise**, representing imperfections of the sensing system such as sensor noise or systematic bias.

The first source affects the true physical parameters of the simulated battery, while the second affects only the observed measurements.

Parameter variability (model uncertainty) In the present work, equivalent circuit model parameters are available as discrete, state-of-charge-dependent data points obtained from parameter identification studies. Rather than using these tabulated values directly, smooth mean parameter curves are constructed and used as the basis for simulation. To this end, the parameter series (R_i and C_i) are fitted in log-space to ensure positivity and to capture multiplicative trends as a function of SoC. The scatter of the original data points around the fitted mean curves is interpreted as natural cell-to-cell variability. This variability is modeled by applying random multiplicative perturbations to the mean parameter trends. The perturbations follow a log-normal distribution, which preserves positivity while introducing realistic proportional deviations across simulated cells. A robust outlier rejection and fitting procedure is employed to estimate both the mean trends and the associated variability. The complete fitting and sampling procedure is described in Appendix A.

Measurement noise (sensor uncertainty) Measurement uncertainty is modeled using a combination of per-sample additive white noise and constant per-trace offset (bias) terms. Both noise components are sampled from zero-mean Gaussian distributions with specified variances. For simulations based on literature parameters, the exact noise magnitudes are not critical. In this setting, noise is introduced primarily to assess the robustness of the inference method and to verify that the model can reliably infer the underlying states in the presence of measurement perturbations. When switching to the model using ECM parameters provided by ABB, the noise magnitudes are adjusted to better reflect realistic experimental conditions. In this case, additive white measurement noise is found to be of minor importance, whereas constant per-trace bias terms represent the dominant source of uncertainty. Accordingly, the variance of the white noise is reduced to a negligible level, while the bias variance is set based on ranges discussed with ABB engineers, reflecting their experience with the experimental setup.

Warm-up initialization. Initializing each simulated dataset from a fully relaxed battery state (i.e. zero voltages across all RC branches) would introduce unrealistic initial transients at the beginning of every sequence. In practical deployment, the battery is typically already in a dynamic state, and the internal RC voltages are rarely exactly zero. To obtain dynamically consistent initial conditions, a short warm-up phase is performed prior to the actual simulation window. A randomly generated current trajectory is first simulated, and the resulting RC branch voltages at the end of this phase are used as the initial conditions for the dataset simulation. To reflect the fact that batteries may occasionally start from a rested state in deployment, 10% of simulated sequences are initialized without warm-up, i.e. from a fully relaxed configuration.

2.2 Current Profiles

To generate voltage observations using the ECM simulator, current trajectories must be provided as inputs. Since the SBI model is trained exclusively on simulated voltage-current data, a method for generating a large number of current profiles is required. These profiles should span the range of operating conditions expected during deployment so that the trained model can generalize to measured data. In this work, several current generation strategies are investigated. We begin with a data-driven approach based on the Karhunen-Loève expansion (KLE), described in Section 2.2.1, which allows the generation of large numbers of statistically similar current trajectories from a small set of reference driving cycles. This provides a convenient baseline for producing statistically realistic excitation patterns at scale. To remove the dependency on measured reference data, a simplified procedural current generator is introduced in Section 2.2.2. This generator combines elementary operating modes commonly observed in practice, such as rest periods, constant charge or discharge currents, and step-like or impulsive load changes. In addition, sinusoidal current profiles are considered in Section 2.2.3. These profiles consist of smooth, periodic charge and discharge currents with varying amplitudes and frequencies. Finally, a degenerate stress-test case consisting of a small constant current is used to examine the behavior of the inference model under deliberately unrealistic excitation, as described in Section 2.2.4. For clarity and reproducibility, concise algorithmic summaries of the current generation procedures used in this work are provided in Appendix B.1.

2.2.1 Karhunen-Loève expansion for stochastic driving cycles

One principled approach to generating realistic and structured current trajectories is based on the Karhunen-Loève expansion (KLE) [8]. The KLE provides a compact, data-driven representation of temporal variability by decomposing a stochastic process into orthogonal modes ordered by their contribution to the signal variance. In this work, the KLE is applied to ensembles of fixed-length current segments extracted from measured driving cycles. Each driving cycle is partitioned into segments of equal duration, which are stacked row-wise into a data matrix.

Let $\mathbf{Y} \in \mathbb{R}^{N \times T}$ denote the resulting data matrix, where each row corresponds to a current segment of duration T. After subtracting the empirical mean function $\mu_I(t)$, the centered matrix $\mathbf{X} = \mathbf{Y} - \mu_I(t)$ is decomposed using a singular value decomposition,

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top,$$

yielding orthonormal temporal modes $\{\phi_n(t)\}$, where each mode corresponds to a row of $\mathbf{V}^\top$ and represents a discretized temporal basis function, along with associated eigenvalues λ_n, which quantify the variance captured by each mode. Any realization of the stochastic current can then be approximated as

$$I(t) \approx \mu_I(t) + \sum_{n=1}^{N} \sqrt{\lambda_n}\, \xi_n\, \phi_n(t), \tag{2.8}$$

where the coefficients ξ_n determine the contribution of each mode. A key property of the Karhunen-Loève expansion is that these coefficients are statistically uncorrelated (and independent under a Gaussian assumption), which enables independent sampling of the ξ_n and thereby the generation of realistic stochastic current trajectories. Truncating the expansion after a small number of modes yields a compact representation that captures the dominant variability of the driving cycles. During simulation, two sampling strategies are considered. In the Gaussian formulation, the coefficients ξ_n are sampled as independent standard normal variables, and the reconstruction follows the classical form above with explicit $\sqrt{\lambda_n}$ scaling. Alternatively, coefficients may be sampled directly from the empirical distribution obtained from the singular value decomposition. In this case, the empirical coefficient matrix is defined as $\mathbf{A} = \mathbf{US}$. Since $\mathbf{A}$ already contains the eigenvalue scaling via the singular values, no additional $\sqrt{\lambda_n}$ factor is applied during reconstruction. This empirical sampling strategy is used for the majority of experiments reported in this work. The specific procedure used to sample KLE-based current trajectories during simulation is summarized in Appendix B.1. During early stages of this work, before experimental current trajectories from ABB were available, baseline driving cycles were taken from the open-access CALCE battery dataset [2], including dynamic profiles such as DST, FUDS, and US06.

2.2.2 Simplified synthetic current generator

A simplified procedural current generator was implemented to assess the sensitivity of the inference model to the choice of current profile generation. The generator produces piecewise-constant current trajectories by randomly combining a small set of elementary operating regimes, including rest periods, constant charge or discharge currents and step-like load changes. The resulting current profiles span a wide range of amplitudes and temporal patterns while remaining straightforward to construct and computationally inexpensive. Unlike data-driven approaches, this method does not rely on measured reference cycles and instead encodes generic assumptions about plausible battery usage scenarios.

2.2.3 Sinusoidal current profiles

Sinusoidal current profiles were considered as an additional class of analytically defined excitation signals. These profiles consist of sinusoidal charge and discharge currents with varying amplitudes and periods. The resulting current trajectories exhibit smooth, periodic temporal patterns. They are not intended to represent realistic driving or usage profiles, but serve to probe the behavior of the inference model under deliberately simplified and structured excitation.

2.2.4 Degenerate current profiles

A degenerate stress-test case was considered in which the ECM is driven by a constant current of small magnitude over the entire simulation horizon. Such a profile provides

minimal temporal variation and is therefore not intended to represent a realistic operating condition. The purpose of this degenerate excitation is to serve as a worst-case baseline for the inference pipeline, in which the applied current significantly differs from what would be expected during operation. This case allows the behavior of the NPE model to be examined under deliberately misspecified input data.

2.3 Definition of Battery State Prior Distributions

For each simulation, latent variables are sampled from predefined prior distributions describing the battery states and measurement conditions. In particular, the battery state is characterized by the state of charge (SoC) and the state of health (SoH). The state of charge represents the current level of stored charge relative to the effective capacity of the battery, typically expressed as a value between 0 and 1. The state of health describes the degradation of the battery relative to a new cell and is commonly defined as the ratio between the current usable capacity and the nominal capacity. The choice of priors depends on the specific experimental setting and is guided by two considerations: physical plausibility and sufficient coverage of the parameter space relevant for the intended evaluation scenario.

State of Charge

In all experiments, the state of charge SoC is sampled from a uniform distribution

$$\mathrm{SoC} \sim \mathcal{U}(0, 1), \tag{2.9}$$

reflecting the absence of prior preference for a particular charge level. This ensures that the inference model is trained across the full operational range.

State of Health

Different prior formulations for the state of health SoH were used depending on the experimental setting.

Gaussian Prior (ABB Experimental Setting) For experiments involving ABB measurement data, the prior was specified directly on the battery capacity. Since the tested cell exhibited capacity variations of ΔQ_{NOM} around its nominal capacity, the capacity was modeled as

$$Q_{\mathrm{eff}} \sim \mathcal{N}(Q_{\mathrm{NOM}}, \sigma_Q^2), \tag{2.10}$$

with $\sigma_Q = \Delta \frac{Q_{NOM}}{2}$, implying that approximately 95% of samples lie within $Q_{\mathrm{NOM}} \pm \frac{\Delta Q_{NOM}}{2}$. This narrower prior reflects the experimentally observed operating regime of the tested cell and focuses the simulation effort on the parameter range relevant for the real measurements.

Uniform Prior for Literature-Based ECM Models For literature-based ECM parameter models in which the state of health is treated as a variable parameter, a broader uniform prior was used

$$\mathrm{SoH} \sim \mathcal{U}(0.6, 1.0). \tag{2.11}$$

This choice ensures that both moderately degraded and near-nominal battery states are sufficiently represented during training, allowing the inference model to learn across a wide range of degradation scenarios. The exact prior ranges used for individual experiments are reported in the corresponding results sections.

2.4 Neural Posterior Estimation

Neural Posterior Estimation (NPE) is a likelihood-free inference method used to approximate the posterior distribution $p(\theta \mid x)$ over model parameters θ given observed data x. Instead of evaluating an explicit likelihood function, NPE trains a neural density estimator to directly learn this conditional mapping from simulated data generated under the prior $p(\theta)$ and the forward model $p(x \mid \theta)$. Once trained, the model can perform inference for a posterior sample conditioned on new observations in a single forward pass, enabling efficient amortized Bayesian inference.

2.4.1 Training Phase

During training, parameter samples θ are drawn from the prior and passed to the simulator to generate synthetic data x. The summary network $h_\psi(x)$ encodes the high-dimensional time-series input (current, voltage and time axis) into a compact latent representation $s = h_\psi(x)$. The posterior network f_ϕ (sometimes called inference network) is implemented as a conditional normalizing flow, which learns an invertible transformation that maps the complex parameter distribution $q_\phi(\theta \mid s)$ into a simple latent base distribution $z \sim \mathcal{N}(0, I)$. It is of note that the approximate posterior distribution $q_\phi(\theta \mid s)$ defined by the learned transformation f_ϕ is just that: an approximation of the true posterior distribution $p(\theta \mid s)$. Training minimizes the negative log-likelihood (NLL) of the true parameters under the learned posterior, optimizing both h_ψ and f_ϕ jointly. This encourages the summary network to extract maximally informative features from the data, while the flow network learns a tractable, invertible representation of the posterior density.

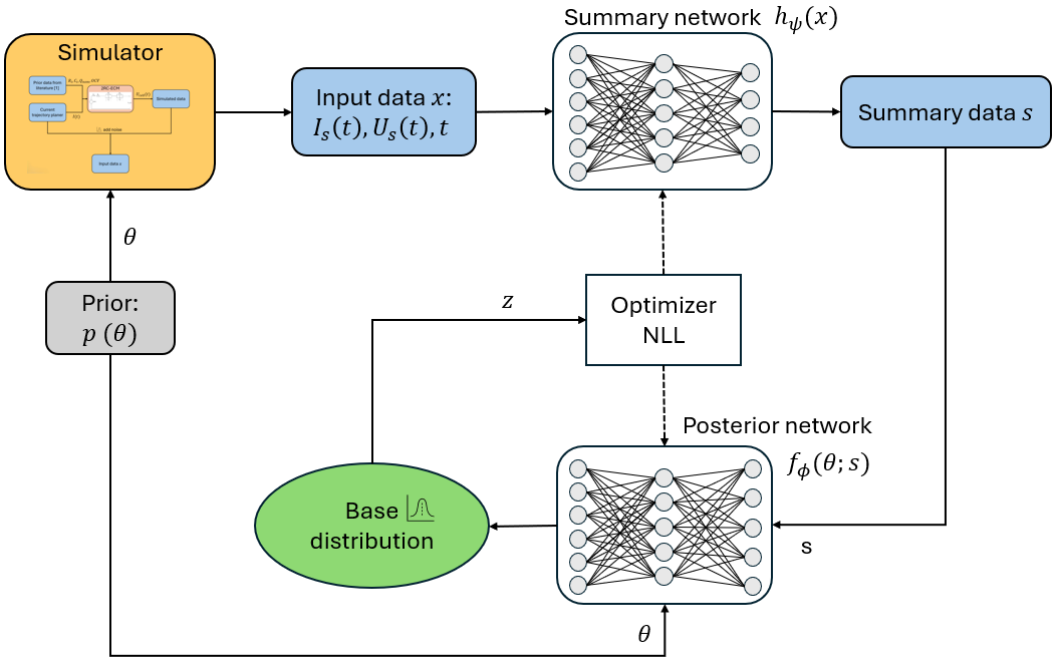


Figure 2.3: Training phase of the Neural Posterior Estimation (NPE) workflow. Prior samples are drawn $\theta \sim p(\theta)$ and passed to the simulator to generate synthetic data $x \sim p(x|\theta)$. The summary network encodes x into a compact latent representation $s = h_\psi(x)$, while the posterior network learns an invertible transformation that maps complex parameter distributions into a simple base distribution in latent space $z = f_\phi(\theta; s)$ where $z \sim \mathcal{N}(0, I)$. Both networks are trained jointly by minimizing the negative log-likelihood (NLL) loss.

2.4.2 Inference Phase

Once trained, the neural posterior estimator can generate posterior samples for new, previously unseen observations without further access to the simulator. An observed current-voltage trajectory x is processed in the same manner as during training to obtain a summary representation s. Conditioned on this representation, posterior samples are generated by drawing latent variables $z \sim \mathcal{N}(0, I)$ and mapping them through the inverse flow transformation $\theta = f_\phi^{-1}(z; s)$. By repeatedly sampling z while keeping the conditioning s fixed, the model produces a set of parameter samples $\{\theta_k\}$ that approximates the posterior distribution $p(\theta \mid x)$. From these samples, point estimates such as posterior means can be computed, while their dispersion provides calibrated uncertainty measures in the form of credible intervals or standard deviations. This procedure enables full Bayesian uncertainty quantification at inference time without requiring explicit likelihood evaluations or re-optimization.

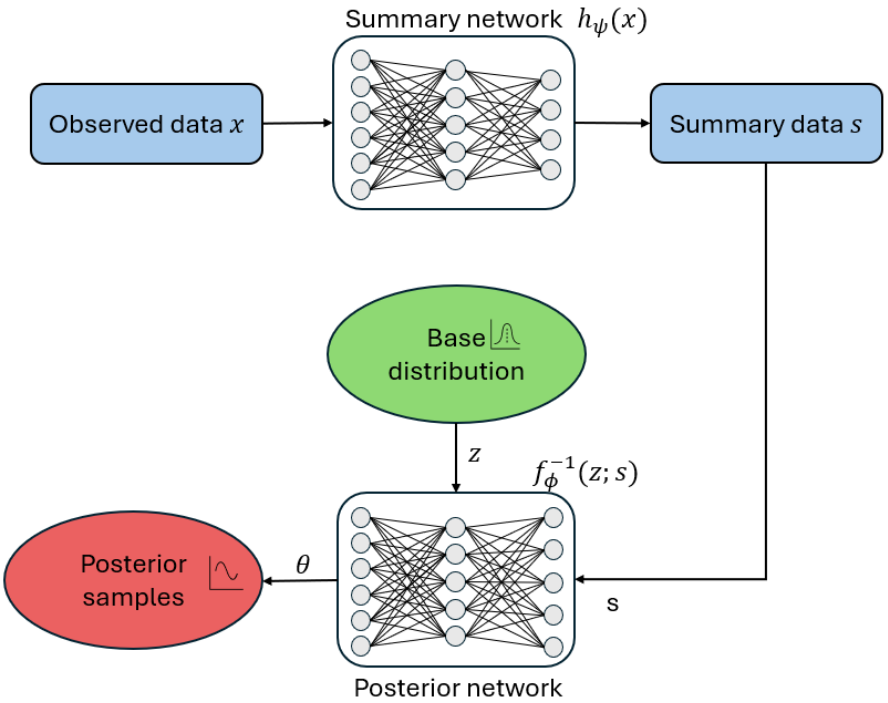


Figure 2.4: Inference phase of the trained Neural Posterior Estimation (NPE) model. Observed data x are first processed by the summary network to obtain a fixed conditioning vector $s = h_\psi(x)$. The posterior network then uses the inverse flow transformation f_ϕ^{-1} to map multiple samples from the base distribution $z \sim \mathcal{N}(0, I)$ into corresponding parameter samples $\theta = f_\phi^{-1}(z; s)$.

2.4.3 Normalizing Flows

The posterior network f_ϕ is implemented as a *normalizing flow*, that is, a sequence of invertible and differentiable transformations that map a simple base distribution into a complex target distribution. Formally, the flow defines an invertible mapping

$$z = f_\phi(\theta; s), \qquad \theta = f_\phi^{-1}(z; s),$$

where the conditional posterior density is obtained via the change-of-variables formula

$$p(\theta \mid s) = p_z(f_\phi(\theta; s)) \left| \det J_{f_\phi}(\theta; s) \right|.$$

During training, the flow parameters ϕ are optimized by minimizing the negative log-likelihood (NLL) of the true parameters under the learned posterior distribution,

$$\mathcal{L}(\phi, \psi) = -\mathbb{E}_{(\theta,x)} \left[\log p_z(f_\phi(\theta; s)) + \log \left| \det J_{f_\phi}(\theta; s) \right| \right],$$

where the expectation is taken over simulated training pairs (θ, x) with summaries $s = h_\psi(x)$. Both the posterior network parameters ϕ and the summary network parameters ψ are optimized jointly through this objective.

In practice, the normalizing flow network is constructed using *CouplingFlow*, as implemented in the BayesFlow framework [7]. In each coupling layer, an invertible transformation is applied only to a subset of the input variables, while the remaining variables are left unchanged and used to condition the transformation. Between successive coupling layers, permutations are applied so that all dimensions are transformed across the full flow depth. BayesFlow provides different choices for the invertible transformation used within each coupling layer. A commonly used option is the affine transformation introduced in RealNVP [9], which applies element-wise scaling and translation to the transformed subset. In this work, the affine transformation is replaced by a more expressive *monotonic rational-quadratic spline* transformation, resulting in a *neural spline flow* [10]. Spline-based coupling layers substantially increase the flexibility of the flow while retaining analytic invertibility and efficient computation of the Jacobian determinant.

2.5 seq-NPE: Sequentially Applied Neural Posterior Estimator

A standard neural posterior estimator (NPE) assumes a fixed prior distribution defined before training (see Section 2.3). Although our workflow supports variable-length input data, the model must be trained on the specific data lengths for which inference is later performed. This requirement quickly becomes impractical. First, an upper bound on the input sequence length must be specified in advance. Second, the architecture of the summary network must be designed to encode time-series data into an informative latent representation for the posterior network. As the input length increases, the summary network must apply increasingly aggressive downsampling while still retaining the relevant information. Designing a single network that handles both very short and very long sequences therefore forces the summary network into a structural compromise. From an application perspective, storing long time-series data solely for inference purposes is also undesirable. In many practical scenarios, inference should be possible using only short, recent measurement windows. To address these limitations, we propose a sequentially applied neural posterior estimator that uses yesterday's posterior as today's prior. The core idea is to process short data segments sequentially: for each segment, the model infers a posterior over the latent states of interest, and this posterior is then used as the prior for the subsequent segment. In practice, this propagation is applied only to a subset of the latent variables, namely the state of charge (SoC) and optionally the state of health (SoH). All remaining parameters are not propagated sequentially but instead retain their original prior distributions at each step. This approach must be clearly distinguished from Sequential Neural Posterior Estimation (SNPE) methods commonly found in the literature. SNPE performs sequential refinement during training by iteratively adapting the simulation proposal distribution and retraining the posterior estimator. In contrast, our approach remains fully amortized and performs sequential updating only at inference time by propagating the posterior distribution between successive data segments. During the course of this thesis, Yang et al. [11] proposed a

related idea for diffusion-based amortized inference models, in which the prior distribution can be adapted at test time without retraining. Their work similarly explores mechanisms for incorporating updated prior information after training. While implemented within a diffusion framework, the underlying motivation is conceptually aligned with our sequential approach, which also seeks to enable flexible prior updating within an amortized inference setting. For the model architecture, we employ the **FusionNet** architecture provided by **BayesFlow** [7]. The sequential inference pipeline is illustrated in Figure 2.5.

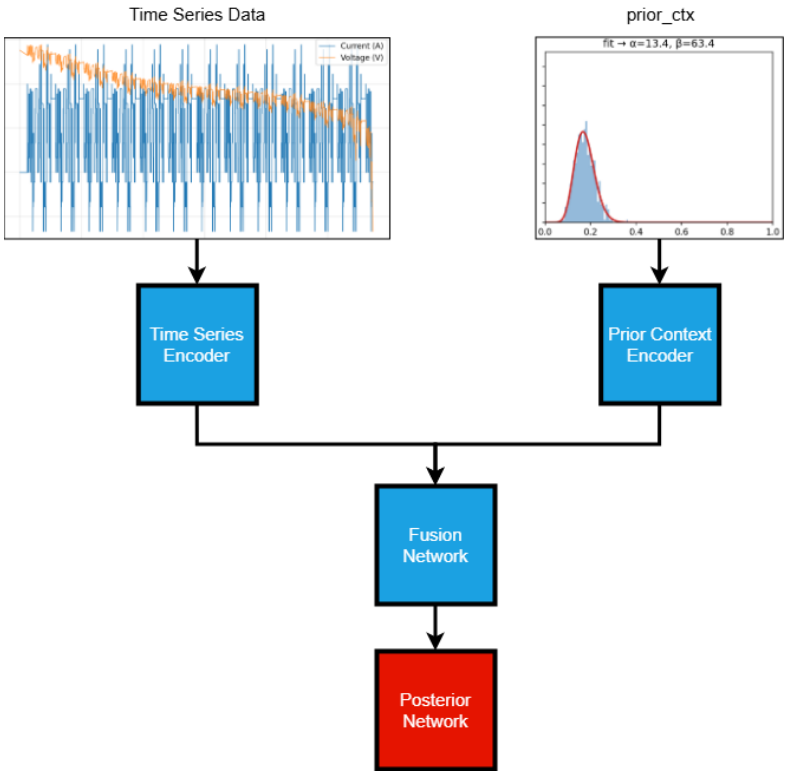


Figure 2.5: Overview of the sequential inference pipeline. The time-series data are encoded using the original summary network. In parallel, the propagated prior distribution is processed by a prior-context encoder. The resulting latent representations are combined through a Fusion Network, and the fused summary is passed to the posterior network to produce the updated posterior distribution. The example time-series data shown were generated from the CALCE battery dataset [2].

2.5.1 Definition of the Prior Context

Two alternative strategies were investigated for representing the propagated posterior as prior context.

1. **Direct sample-based representation.** The posterior samples are provided directly as input to the model. Since posterior samples form an unordered set,

a permutation-invariant architecture is required. For this purpose, we employ a **SetTransformer** encoder, which is specifically designed to process sets while preserving permutation invariance. This allows the network to learn a flexible representation of the full posterior sample distribution.

2. **Parametric summary representation.** Instead of passing the samples directly, a low-dimensional parametric summary is derived from them. Specifically, a Beta distribution is fitted to the posterior samples using the method of moments. From the estimated shape parameters (α, β), two summary statistics are computed:

   - The mean $m = \frac{\alpha}{\alpha+\beta}$,
   - The concentration parameter $\kappa = \alpha + \beta$.

   The concentration parameter increases with decreasing posterior uncertainty and therefore serves as a proxy for confidence. To ensure numerical stability in the posterior network, κ is capped at a predefined upper bound. Since this representation is already low-dimensional, the corresponding prior-context encoder is implemented as a simple linear projection rather than a deep summarization network.

2.5.2 Generating the Prior Context

Two alternative strategies were investigated for generating the prior context during training of the sequential NPE.

Empirical approach In the empirical approach, a previously trained standard NPE is used to generate posterior samples from the first half of a simulated time-series dataset. These samples are then used to construct the prior context for the sequential model. Depending on the chosen representation (see Section above), the samples are either:

- provided directly to the prior-context encoder, or
- summarized into a low-dimensional representation consisting of the mean and $\log(\kappa)$.

This approach mimics the intended deployment setting, where the posterior from a previous data segment is propagated forward and used as the prior for the next segment.

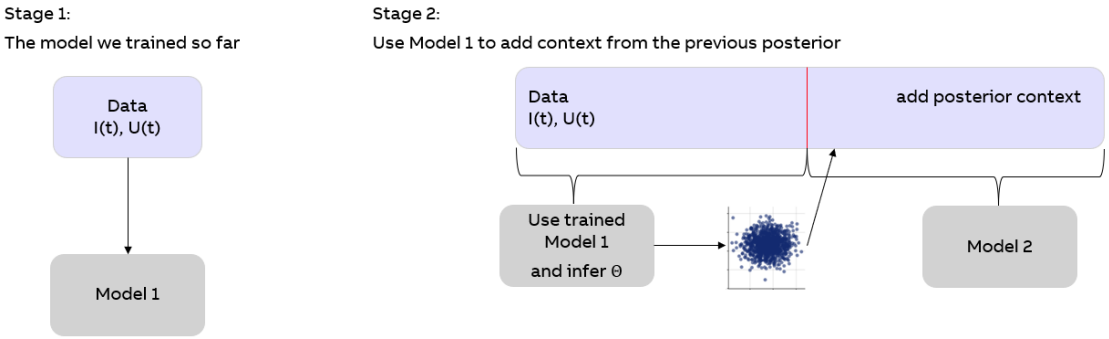


Figure 2.6: Empirical prior-context generation. A standard NPE is first trained and subsequently used during training of the sequential model to provide posterior samples serving as prior context.

Synthetic approach In the synthetic approach, the prior context is generated directly from a parametric distribution without relying on a pretrained NPE. Specifically, a Beta distribution is sampled with lower bounds of 0.5 on both shape parameters and an upper bound of $\alpha + \beta \leq 2000$ to control concentration. From this sampled distribution, a ground-truth initial SoC is drawn. This initial state is then used to simulate a feasible current trajectory, which in turn determines the corresponding terminal state of charge for the dataset. This procedure eliminates the need for a pretrained NPE during training of the sequential model and allows full control over the diversity and concentration of prior distributions encountered during training.

2.5.3 Inference Phase

Figure 2.7 illustrates the deployment strategy of the sequential model. For the first segment of measured data, the sequential NPE is provided with a uniform prior context, as no prior information from previous segments is available. From the second segment onward, the posterior distribution inferred from the previous segment is propagated and supplied as the new prior context. In this way, the model performs recursive Bayesian updating across successive data segments.

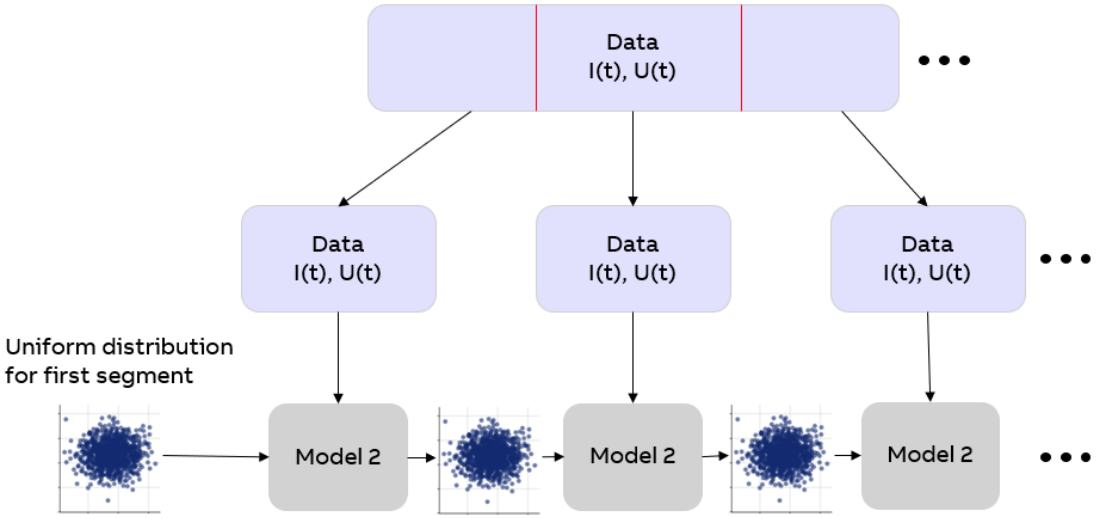


Figure 2.7: Deployment of the sequential inference model. The posterior distribution obtained from one segment is used as the prior for the subsequent segment.

2.6 Diagnostics and Validation

In simulation-based inference, classical dataset splitting strategies are insufficient for model validation. Both training and validation data are generated by the same simulator and therefore share the same modeling assumptions. As a result, strong performance on held-out simulated data does not guarantee that the simulator-inference pipeline is statistically correct or physically meaningful. If the simulator itself is misspecified, the inference model may nevertheless appear to perform well when evaluated only on synthetic data.

For this reason, simulation-based inference requires dedicated diagnostics that go beyond predictive accuracy on simulated data. Simulation-based calibration (SBC) provides such a diagnostic by testing whether the inference algorithm produces statistically consistent posterior distributions when the data are generated from the assumed simulator. SBC therefore validates the internal statistical correctness of the simulator-inference pipeline under the modeling assumptions.

However, SBC does not assess robustness to model misspecification or generalization beyond the simulator. Consequently, validation on experimentally measured data remains essential to evaluate practical performance and real-world applicability.

2.6.1 Simulation-based Calibration - SBC

Simulation-based calibration (SBC) is a consistency check for simulation-based inference methods, originally introduced by [12] and commonly used in modern SBI benchmarks [13]. The core idea of SBC is to verify that the inferred posterior distribution is statistically consistent with the generative model used during training.

In SBC, parameters are repeatedly drawn from the prior, $\theta \sim p(\theta)$, synthetic observations are generated via the simulator, $x \sim p(x \mid \theta)$, and posterior samples are obtained by running inference, $\theta' \sim q(\theta \mid x)$. If the approximate posterior $q(\theta \mid x)$ is exact, then the distribution of θ' marginalised over all simulated datasets is equal to the prior distribution, $\theta' \sim p(\theta)$. Equivalently, the rank of the true parameter value among posterior samples should be uniformly distributed.

In practice, SBC is assessed by computing posterior ranks for each inferred parameter across many simulated datasets and analysing their empirical distribution. Deviations from uniformity indicate systematic inference errors such as bias, overconfidence, or underconfidence. We visualise SBC results using rank histograms, empirical cumulative distribution functions (ECDFs), and empirical coverage curves, which provide complementary diagnostics of different failure modes. Detailed interpretations of these visualizations can be found in [14].

SBC is a powerful diagnostic for detecting inconsistencies between the simulator and the inference procedure. However, it should be interpreted as a consistency check rather than a performance metric: for example, a posterior that collapses to the prior would trivially pass SBC while being uninformative [13]. For this reason, SBC is complemented by posterior predictive checks in data space and by validation on real measured data.

2.6.2 Posterior Predictive Check - PPC

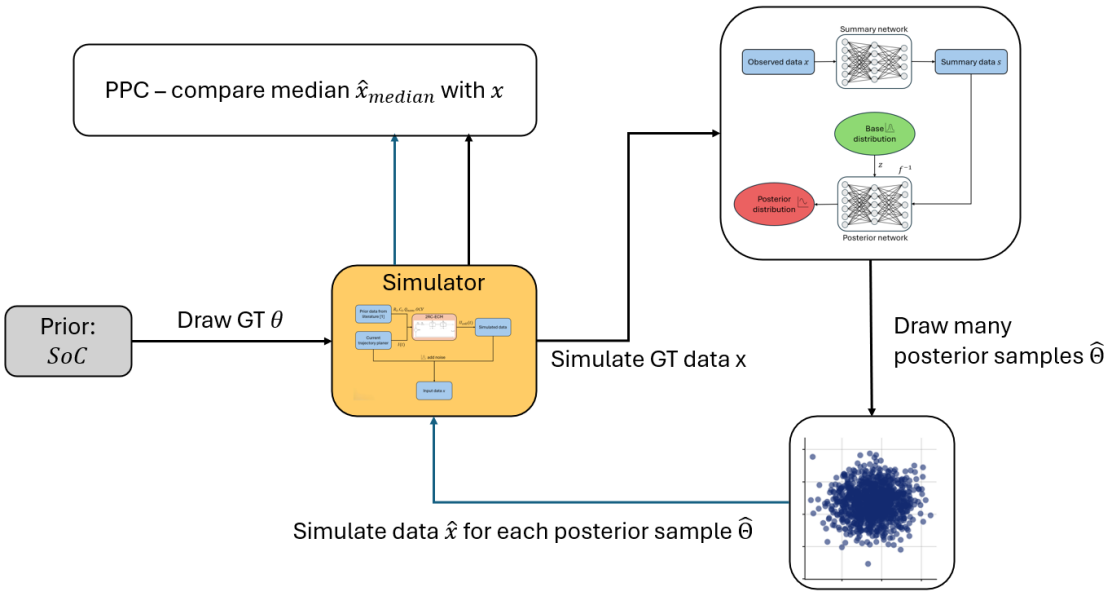


Figure 2.8: Schematic illustration of the posterior predictive check (PPC) procedure used in this work. First, a prior sample θ is drawn from the prior and used to generate a synthetic dataset $x \sim p(x \mid \theta)$ via the simulator. Conditioned on this dataset, the trained neural posterior estimator produces posterior samples $\hat{\theta} \sim p(\theta \mid x)$. These posterior samples are then passed through the same simulator to generate posterior predictive replicates $\hat{x} \sim p(x \mid \hat{\theta})$. Finally, discrepancies between the ground-truth data x and the posterior predictive replicates $\hat{x}$ are evaluated using suitable summary statistics.

Posterior predictive checks (PPCs) assess whether observed data are plausible under the posterior predictive distribution implied by the model and inferred parameters. If the model provides an adequate description of the data-generating process, then replicated data simulated from the posterior predictive distribution should resemble the observed data in relevant aspects [15, Chapter 6.3]. Formally, given observed data x_o and an approximate posterior $q(\theta \mid x_o)$, the posterior predictive distribution is

$$p(x' \mid x_o) = \int p(x' \mid \theta)\, q(\theta \mid x_o)\, d\theta.$$

Posterior predictive checking is therefore a self-consistency check: systematic discrepancies between observed data and posterior predictive samples indicate model misspecification, while isolated discrepancies may occur due to chance.

In this work, posterior predictive checks are implemented by drawing replicated datasets x' from the posterior predictive distribution and comparing them directly to the observed data in data space. Following common practice in simulation-based inference benchmarks [13], we quantify posterior predictive fit using the median ℓ_2 distance

between the observed time series and samples drawn from the posterior predictive distribution. The median is used instead of the mean to reduce sensitivity to outliers. This diagnostic provides an intuitive data-space comparison that complements parameter-space diagnostics such as simulation-based calibration. Posterior predictive checks do not assess posterior calibration or uncertainty quantification and should therefore not be interpreted as a standalone performance metric, but rather as a sanity check for model adequacy.

2.6.3 Posterior Contraction and Z-Score Diagnostics

Beyond calibration diagnostics, we analyse two additional quantities that provide dataset-level insight into estimation accuracy and informativeness.

Z-score: For a parameter θ with ground-truth value $\theta^\star$, posterior mean μ_θ, and posterior standard deviation σ_θ, the z-score is defined as

$$z = \frac{\mu_\theta - \theta^\star}{\sigma_\theta} \tag{2.12}$$

The z-score measures the estimation error in units of posterior uncertainty. Values $|z| \approx 0$ indicate accurate and well-calibrated inference, whereas large $|z|$ indicate bias or underestimated uncertainty.

Posterior Contraction: Posterior contraction quantifies the reduction of uncertainty relative to the prior:

$$C = 1 - \frac{\mathrm{Var}(\theta \mid x)}{\mathrm{Var}(\theta)} \tag{2.13}$$

A value $C \approx 0$ indicates that the dataset is uninformative, as the posterior variance remains close to the prior variance. In contrast, $C \to 1$ corresponds to strong information gain and substantial uncertainty reduction.

Chapter 3

Results

3.1 Joint State-of-Charge and State-of-Health Estimation

In this section, we present the results of a neural posterior estimator (NPE) trained using a two-RC Thevènin equivalent-circuit model (ECM) with parameters taken from [1]. The model was trained on simulated datasets of varying observation lengths $T \in [500\,\mathrm{s}, 8000\,\mathrm{s}]$, sampled at 1 Hz. The current trajectories used for simulation were generated using the Karhunen-Loève expansion (KLE) described in Section 2.2.1, based on current data from [2]. The objective of this experiment is joint estimation of the terminal state of charge (SoC) and the state of health (SoH). The prior distributions for the latent states are defined as:

$$\mathrm{SoC} \sim \mathcal{U}(0, 1), \tag{3.1}$$

$$\mathrm{SoH} \sim \mathcal{U}(0.6, 1.0). \tag{3.2}$$

Figure 3.1 shows the ECDF difference plot for simulation-based calibration (SBC). The empirical rank distribution lies within the expected confidence bands, indicating that the posterior is well calibrated under the generative model.

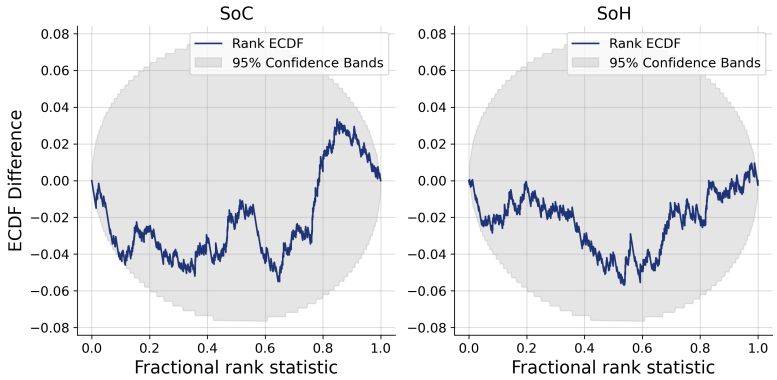


Figure 3.1: Simulation-based calibration (SBC) ECDF difference plot for joint SoC-SoH estimation under the literature-based ECM.

Figure 3.2 presents posterior recovery plots for SoC and SoH. Perfect recovery corresponds to all points lying on the diagonal, indicating agreement between posterior median and ground truth. The model achieves highly accurate recovery for SoC across the entire range. In contrast, SoH estimation exhibits substantially larger dispersion, indicating greater uncertainty and reduced identifiability.

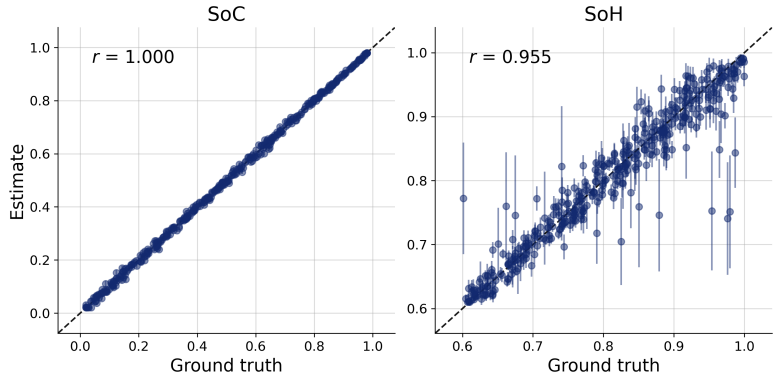


Figure 3.2: Posterior recovery plots for joint SoC-SoH estimation. Left: SoC. Right: SoH. The dashed diagonal indicates perfect recovery.

The reason for the increased difficulty in estimating SoH becomes evident when examining Figure 3.3. The horizontal axis represents posterior contraction, defined as the reduction in uncertainty relative to the prior, while the vertical axis shows the corresponding Z-score (More under section: 2.6.3). Each data point corresponds to a simulated dataset, and the colour gradient indicates the total change in SoC within that dataset.

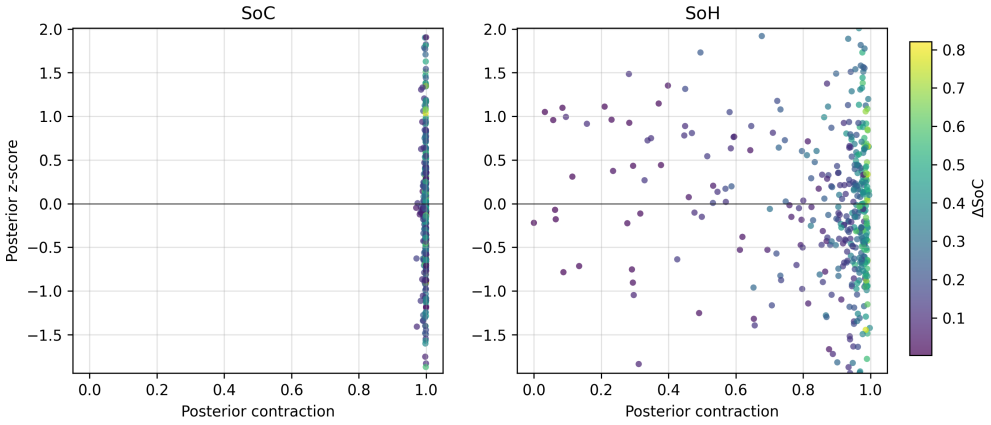


Figure 3.3: Z-score versus posterior contraction for joint SoC (left) and SoH (right) estimation. Each point corresponds to one simulated dataset and is coloured according to the total SoC excursion ΔSoC. While SoC shows consistently high contraction, the SoH panel reveals a visible trend of increasing contraction with larger ΔSoC.

A clear trend is observable: datasets exhibiting larger ΔSoC yield stronger posterior contraction for SoH. In other words, the greater the SoC excursion within a measurement window, the more informative the dataset becomes about the battery's SoH. The dependence of SoH identifiability on ΔSoC can be understood directly from the system dynamics. SoH enters the equivalent circuit model primarily through the effective capacity Q_{Ah}. When two batteries with different capacities are subjected to the same current trajectory, their SoC trajectories evolve at different rates. A battery with lower capacity experiences a larger change in SoC for the same transferred charge. If both batteries start from the same initial SoC, their terminal voltages may initially coincide. However, as charge accumulates or depletes, their SoC trajectories gradually diverge. Because the open-circuit voltage depends nonlinearly on SoC, this divergence causes a systematic drift in the voltage response. The magnitude of this divergence increases with the total charge throughput within the observation window. Consequently, datasets with larger total SoC excursion ΔSoC amplify the observable effect of capacity differences. In contrast, if the SoC changes only marginally within a measurement window, the voltage responses of batteries with different SoH remain nearly indistinguishable and can be masked by measurement noise. The observed relationship between posterior contraction and ΔSoC therefore reflects a structural identifiability property of the system. Capacity differences become observable only when sufficient charge is transferred.

3.2 SoC Estimation on ABB Data Using a 3RC Thevènin ECM

This section evaluates the proposed NPE approach in a realistic experimental setting. In contrast to the previous section, which considered joint SoC and SoH estimation using a literature based 2RC ECM, we now focus on terminal SoC estimation with a 3RC ECM. For this study, ABB provided experimentally identified parameters of the ECM model for the tested battery cell. The parameter values for this ECM cannot be disclosed as they are company secrets. The objective is to assess whether an NPE model trained purely on synthetic data can generalize to experimentally measured current and voltage trajectories.

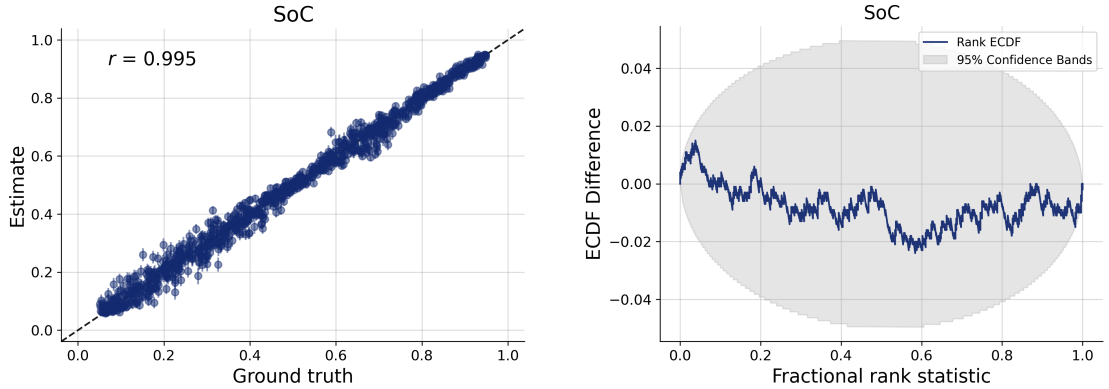
We first validate posterior recovery and calibration of the model on simulated data. Subsequently, the model is applied to real measurement data provided by ABB. These measurements were previously analysed using a multi scale dual Kalman filtering approach [16], enabling a comparison against an established estimation framework and providing a test of generalization.

3.2.1 Validation on Simulated Data

We first assess the inference performance of the trained NPE model on simulated data generated from the forward model. In this setting, the true parameter values used to generate the synthetic observations are known, allowing direct evaluation of posterior recovery accuracy and uncertainty calibration. Figure 3.4 shows inference results on

simulated data. The posterior median closely tracks the ground-truth SoC in the vast majority of cases. This confirms that the trained NPE model is capable of learning the inverse mapping defined by the simulator.

To assess posterior calibration, Figure 3.4b shows the ECDF difference plot for the terminal SoC. The empirical cumulative distribution function lies well within the 95 % confidence bands, indicating that the model is calibrated.



(a) Posterior recovery on simulated data.

(b) SBC ECDF difference plot.

Figure 3.4: Validation on simulated data. **(a)** Posterior recovery for the terminal state of charge SoC; each point corresponds to one simulated dataset, and the dashed line indicates perfect recovery. **(b)** Simulation-based calibration (SBC) ECDF difference plot for SoC with 95 % confidence bands.

3.2.2 Inference on Experimental ABB Data

To evaluate generalization beyond the training simulator, the NPE model is applied to real current and voltage measurements obtained from laboratory experiments conducted by ABB. The experiments were performed under approximately constant temperature conditions close to 25 °C. The measured current profile was used only to construct a Karhunen-Loève expansion (KLE), from which a low-dimensional stochastic current model was derived. During training, synthetic current trajectories were generated by sampling from this KLE-based model, ensuring that the statistical structure of the training currents matches that of the experimental data without directly using the measured current signal itself.

Figure 3.5 presents the inference results over the full experimental trajectory. The first panel shows the measured current, while the remaining panels display the inferred SoC posterior obtained with observation window lengths of $T = 300\,\text{s}$, $600\,\text{s}$, and $900\,\text{s}$ (sampling rate of 10 Hz). The blue curve denotes the posterior median, the orange bars indicate the 68 % credible interval, and the black dashed line represents the ground-truth SoC.

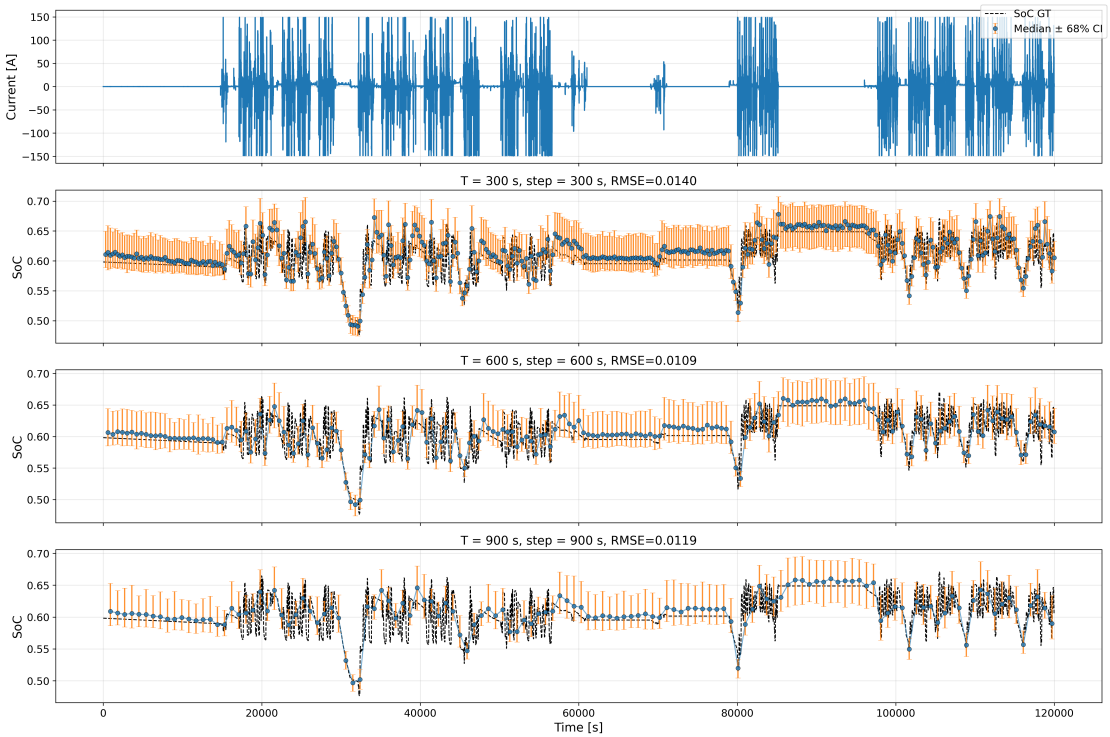


Figure 3.5: Real current and voltage measurements and corresponding NPE inference results over the full experimental trajectory. Panels three to five show inferred SoC posteriors for observation windows of 300 s, 600 s, and 900 s, respectively. Blue markers indicate the posterior median, orange bars denote the 95 % credible interval, and the black dashed line represents the ground-truth SoC.

Due to the long duration of the experimental dataset (33h), it is difficult to visually assess posterior concentration and uncertainty reduction over the full trajectory. To facilitate a more detailed comparison of inference precision, the following subsection examines a representative subsection of the data.

3.2.3 Effect of Observation Window Length

Figures 3.6-3.8 show inference results for a subsection of the data shown in Figure 3.5. For observation windows of 300 s, 600 s, and 900 s, the posterior credible intervals remain of comparable width, indicating that uncertainty does not decrease substantially with increasing window length in this regime. A slight improvement in point estimation accuracy is observed when increasing the window length from 300 s to 600 s, where the RMSE decreases from 0.0148 to 0.0102 on this particular dataset. However, extending the window further to 900 s does not yield additional improvement, with RMSE remaining at a similar level as for 600 s.

Across multiple evaluated segments, the differences in RMSE between these window lengths remain small and are not fully consistent, as illustrated in Figure 3.5. The

smallest RMSE is obtained for $T = 600\,\mathrm{s}$, followed closely by $T = 900\,\mathrm{s}$, while $T = 300\,\mathrm{s}$ performs slightly worse. Nevertheless, all values lie within a narrow range between approximately 0.0108 and 0.0139, suggesting that segment length in this range is not a dominant factor for estimation performance.

In contrast, preliminary experiments with very short windows, such as 4 s or 8 s, showed a clear degradation in estimation quality, which is expected given the limited information content of such short trajectories.

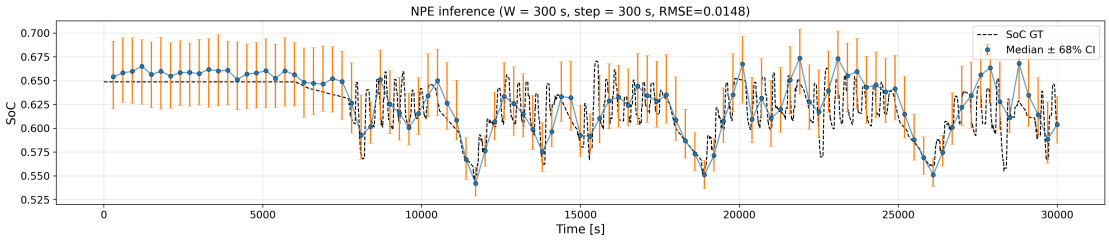


Figure 3.6: Zoomed-in NPE inference results on a representative subsection of the ABB experimental data using an observation window of 300 s.

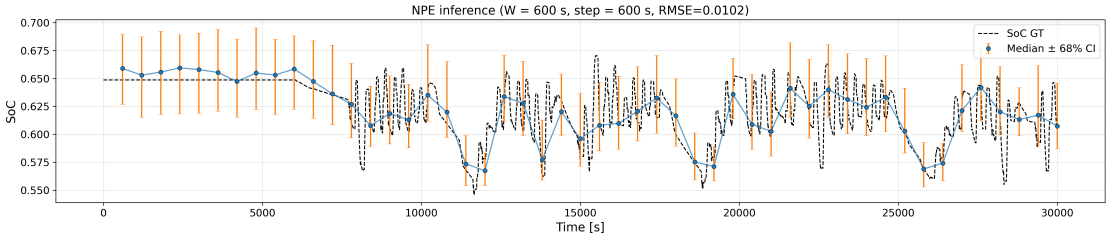


Figure 3.7: Zoomed-in NPE inference results on the same data subsection as in Figure 3.6, using an observation window of 600 s.

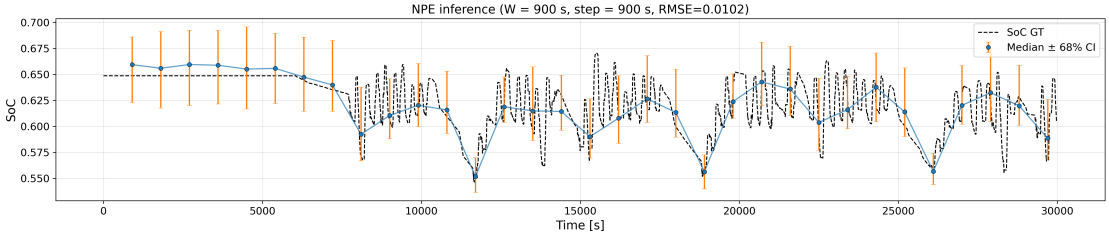


Figure 3.8: Zoomed-in NPE inference results on the same data subsection as in Figures 3.6 and 3.7, using an observation window of 900 s.

3.2.4 Robustness to Current Model Misspecification

So far, we have demonstrated that it is possible to train an NPE model using an ECM simulator driven by currents generated via a Karhunen–Loève expansion. However,

constructing a KLE requires measured current data. This could represent a potential limitation of the NPE approach for battery state estimation if the model were highly sensitive to current misspecification. In practical deployment, operating currents may differ substantially from those used to construct the KLE. If the NPE were unable to infer states under such deviations, the approach would require extensive collections of realistic operating currents for each application scenario, which could limit its practicality. This concern is particularly relevant for electric vehicle applications, where current profiles depend strongly on driver behavior and environmental conditions.

To investigate this issue, we evaluated the performance of the NPE when trained using alternative, non data-driven current generation strategies. Specifically, we compared the NPE performance on the same dataset for four different current generation approaches:

- Karhunen-Loève expansion derived currents,
- a simplified procedural current generator combining rest periods, constant charge or discharge phases, step changes, and impulsive load profiles,
- sinusoidal current profiles,
- degenerate constant current profiles.

Representative examples of the four current generation strategies are shown in Figure 3.9.

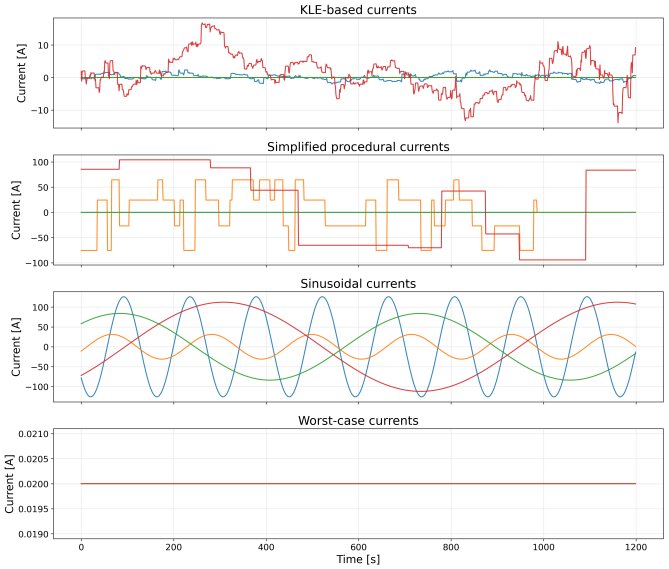


Figure 3.9: Representative examples of the four current generation strategies used for robustness analysis: (top) Karhunen–Loève expansion (KLE) based currents, (second) procedurally generated synthetic currents, (third) sinusoidal current profiles, and (bottom) degenerate constant current profiles. Each panel shows multiple realizations.

Details of the respective current generation procedures are provided in Section 2.2. The corresponding inference results are shown in Figures 3.10-3.13.

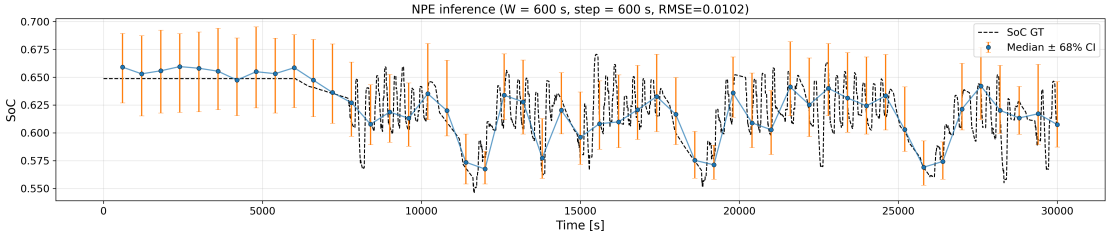


Figure 3.10: Inference results of a NPE trained using Karhunen-Loève expansion derived currents.

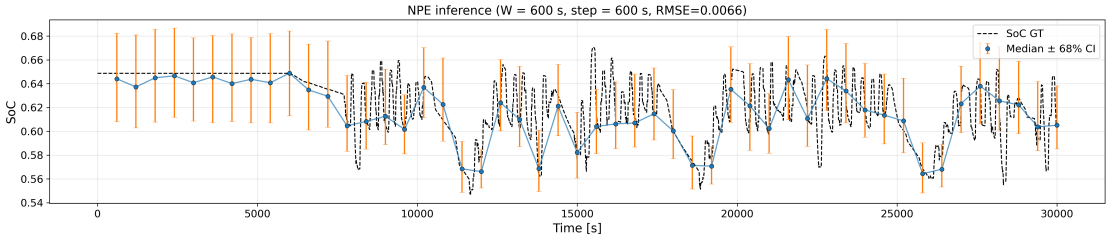


Figure 3.11: Inference results of a NPE trained using a procedural current generator combining rest periods, constant charge or discharge phases, step changes, and impulsive load profiles.

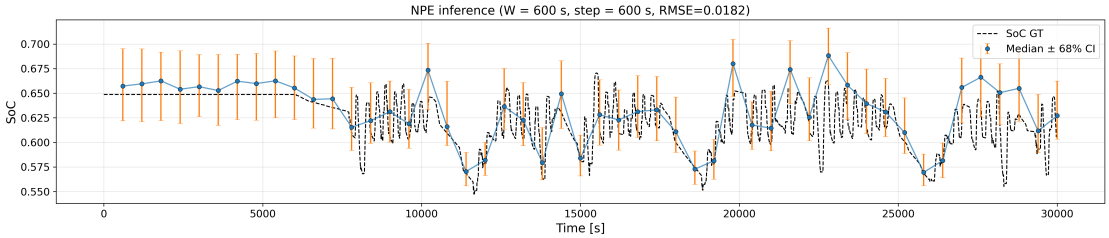


Figure 3.12: Inference results of a NPE trained using sinusoidal currents.

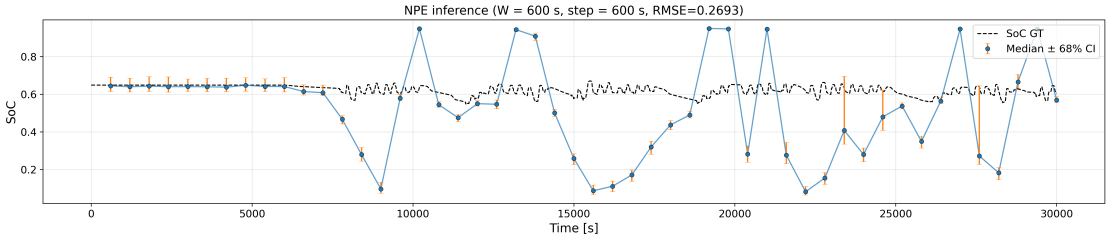


Figure 3.13: Inference results of a NPE trained using degenerate constant currents.

The results indicate that the NPE is largely robust to moderate current misspecification. The RMSE values obtained when training with KLE based currents, simplified procedural currents, and sinusoidal currents are very similar, as shown in Figures 3.10, 3.11, and 3.12.

In contrast, a substantial degradation in performance is observed for the degenerate current generator. When trained using a constant current of amplitude 0.2 A, the NPE provides accurate predictions during rest periods but fails during load phases. This behavior is expected, as the model was not exposed to dynamic load currents during training and therefore did not learn the influence of internal resistances and capacitances in the ECM.

These findings suggest that, while the NPE does require exposure to representative excitation patterns during training, it is not necessary to use highly realistic or data-specific current models. Rather, it is sufficient to train the model using general current profiles that include both load and rest periods to capture the essential system dynamics.

3.2.5 Robustness to Temperature-Induced OCV Mismatch

An additional experiment was performed to evaluate robustness to model mismatch arising from temperature-dependent variations in the OCV-SoC relationship. Importantly, temperature itself is not modeled explicitly in our NPE framework. Instead, the mismatch is induced indirectly by applying an NPE trained using ECM parameters identified at 25 °C to data recorded at 45 °C.

Despite this deliberate mismatch in model parameters, overall SoC estimation performance remained strong. The posterior median continues to track the reference trajectory closely over most of the operating range, indicating substantial robustness to moderate parameter shifts.

A more detailed inspection of Figure 3.14 reveals that the estimation accuracy is not uniform across the entire SoC range. During the initial rest period at approximately $\mathrm{SoC} \approx 0.3$, the inferred SoC remains highly accurate. In contrast, larger deviations are observed during subsequent rest phases around $\mathrm{SoC} \approx 0.65$.

This behavior aligns with the known temperature dependence of the measured OCV-SoC characteristics of the cell. Experimental OCV data indicate that the difference between the 25 °C and 45 °C OCV curves is locally largest in the region $\mathrm{SoC} \in [0.6, 0.7]$. In this regime, temperature-induced voltage shifts are more pronounced, leading to systematic offsets that cannot be fully compensated by an inverse model trained exclusively on 25 °C parameters.

This behavior is physically consistent. Because SoC inference relies strongly on the nonlinear OCV-SoC relationship, discrepancies in this mapping directly affect identifiability. The results therefore demonstrate that the method exhibits structured sensitivity to OCV parameter variation rather than arbitrary degradation. Performance deterioration occurs predominantly in SoC regions where temperature-induced OCV differences are largest, indicating that the inference mechanism remains interpretable under moderate parametric mismatch.

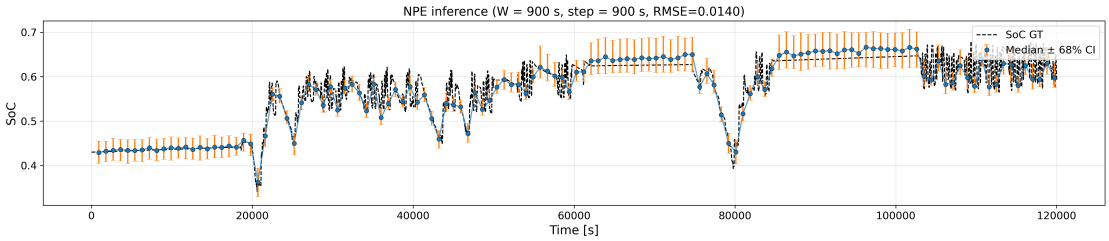


Figure 3.14: SoC inference using an NPE trained with ECM parameters identified at 25 °C and applied to data measured at 45 °C.

3.2.6 Comparison with Kalman Filter Approach

To provide a reference against an established estimation method, we compare the NPE-based SoC estimates with results obtained from the Dual Extended Kalman Filter (DEKF) approach reported in [16]. The DEKF method was previously applied to the same experimental dataset by ABB and therefore provides a suitable benchmark. For this comparison, we use the same NPE model as in Figure 3.5, which was trained on synthetic data generated using the KLE-based current model described in Section 2.2.

Figure 3.15 and Figure 3.16 show two representative segments extracted from the full experimental trajectory presented in Figure 3.5. In each segment, the SoC inferred by the NPE model is compared with the DEKF estimate and the reference SoC trajectory. These localized views allow a clearer visual comparison of the dynamic behavior of the two estimation approaches, highlighting differences in tracking accuracy and local deviations that are difficult to observe in the full 33-hour trajectory. Across both segments, the DEKF exhibits consistently tighter tracking of the reference trajectory, which is also reflected in a lower root mean squared error (RMSE) compared to the NPE model (0.0021 vs. 0.0151). It should be noted that the RMSE is computed only for time steps at which both models provide a prediction.

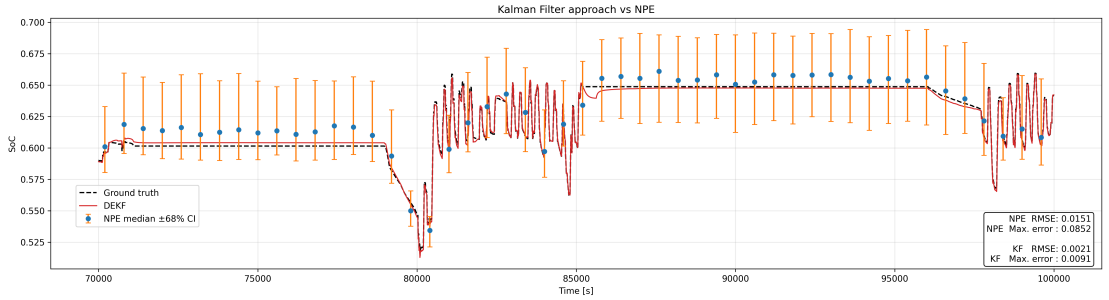


Figure 3.15: Comparison of NPE-based SoC inference and the DEKF estimate on a representative segment of the experimental dataset shown in Figure 3.5.

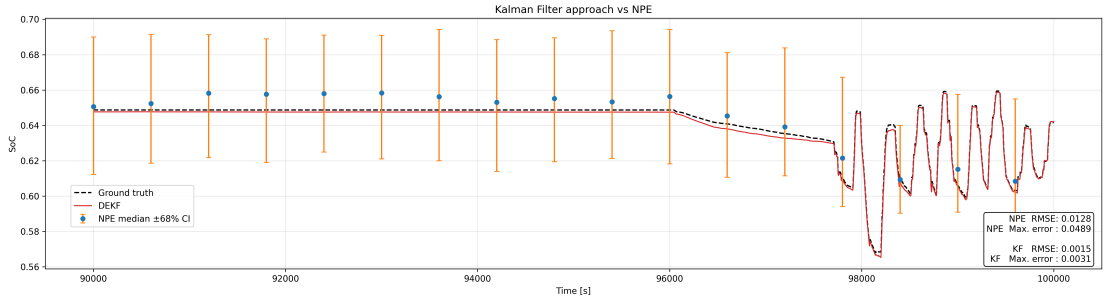


Figure 3.16: Comparison of NPE-based SoC inference and the DEKF estimate on a second representative segment of the experimental dataset shown in Figure 3.5.

3.3 seq-NPE: Performance and Comparison

This section evaluates the proposed sequentially applied NPE (seq-NPE) introduced in Section 2.5 and compares it against two reference models: an all-data NPE (ad-NPE) and a short-window NPE (sw-NPE). The objective is to assess whether propagating posterior information between successive data segments improves inference compared to a fixed short-window baseline. The ECM parameters used in this section correspond to the literature-based identification reported by [1].

3.3.1 Evaluation Setup

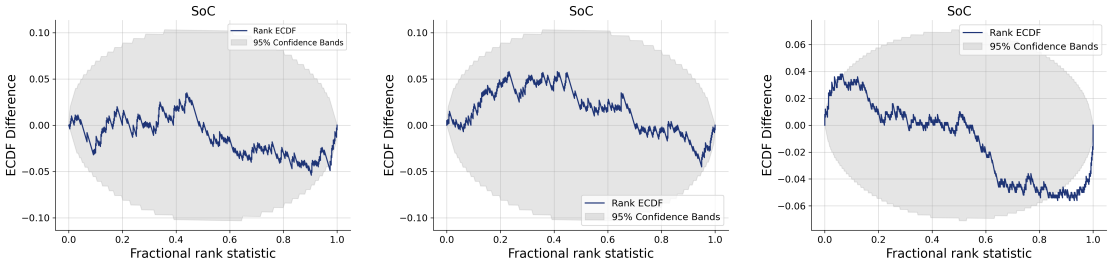
We compare three inference paradigms:

- **ad-NPE (all-data NPE):** A model which receives the complete measurement history from the beginning of the trajectory up to the current prediction time (growing window). This model serves as an approximate upper bound, as it has access to all available information.
- **sw-NPE (short-window NPE):** A model which receives only the most recent 120 s of time-series data. This represents the practical baseline where long histories are not stored.
- **seq-NPE (sequential NPE):** A model which receives a 120 s time-series window together with a propagated prior context derived from the posterior of the previous window. Multiple seq-NPE variants are evaluated, differing in prior representation and prior-context generation strategy as described in Section 2.5.

For all models, inference is performed on simulated test trajectories which are segmented into consecutive, non-overlapping windows of length 120 s. The simulated current trajectories in this evaluation are generated using the simplified procedural current model described in Section 2.2.2. For each window, the model predicts a posterior distribution over the latent state at the end of the window. In the sequential setting, the

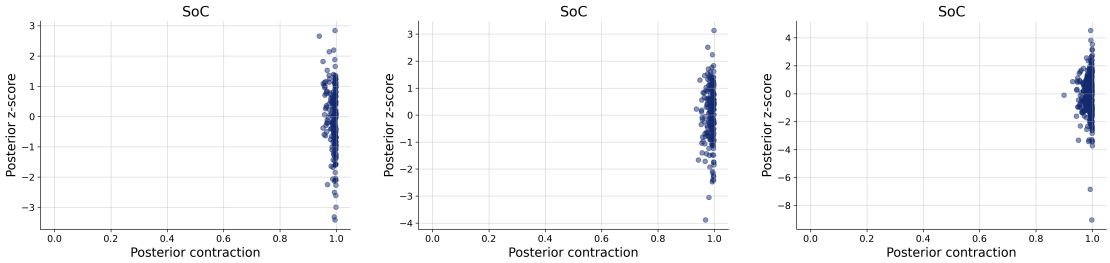
posterior from window t is propagated as prior context to window $t+1$, while the first window is evaluated with a uniform prior.

3.3.2 SBC Results



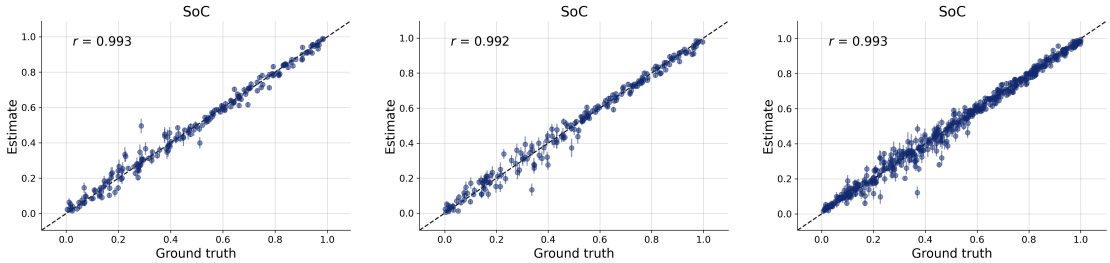
(a) ad-NPE **(b)** sw-NPE **(c)** seq-NPE

Figure 3.17: Simulation-based calibration (SBC) ECDF difference plots for the evaluated models. The seq-NPE shown here corresponds to the synthetically trained variant using the parametric prior representation $[\mu, \log(\kappa)]$. The shaded region indicates the expected variability under perfect calibration. Additional SBC results for alternative seq-NPE variants are provided in Appendix C.



(a) ad-NPE **(b)** sw-NPE **(c)** seq-NPE

Figure 3.18: Z-score over posterior contraction. It can be observed that for all models a 120s dataset is very informative (post contraction approx = 1).



(a) ad-NPE **(b)** sw-NPE **(c)** seq-NPE

Figure 3.19: Recovery on simulated data. It can be observed that all models perform similarly.

Figure 3.17 shows SBC ECDF difference plots for the three models. The ad-NPE and sw-NPE curves remain close to the zero line and lie well within the expected variability bands, indicating approximately uniform rank statistics and good calibration. In contrast, the shown seq-NPE variant exhibits a weak but consistent wave-like deviation (positive on the left and negative towards the right tail), which is characteristic of mild overconfidence (i.e., under-dispersed posteriors). While the curve mostly remains within the variability bands, it intermittently approaches the band boundaries. SBC plots for additional seq-NPE variants (Appendix C) show the same qualitative tendency.

Figure 3.18 shows the distribution of standardized z-scores, defined as $z = (\theta_{\text{true}} - \mu_{\text{post}})/\sigma_{\text{post}}$. For all models, the bulk of the mass is concentrated around zero, with most values lying in the interval $[-1, 1]$, indicating that posterior means are typically within one posterior standard deviation of the ground truth. This suggests that the overall uncertainty scaling is reasonable. At the same time, the posterior contraction being close to 1 indicates that the observation window is highly informative in all cases, meaning that the models can determine the SoC with high precision from their respective input data. A small number of outliers with $|z| \approx$ 3-4 are observed, indicating occasional underestimation of uncertainty; however, these cases are not dominant and do not substantially distort the overall distribution. No strong systematic bias is apparent across models.

Finally, Figure 3.19 presents parameter recovery results on simulated data. For all models, posterior medians are closely aligned with the ground-truth values and concentrate tightly around the identity line, indicating accurate recovery in the considered setting. While a small number of outliers are visible, the majority of predictions lie very close to the true parameter values. Differences between architectures are marginal, with no consistent performance gap across models. The recovery experiments are conducted at a fixed observation length; all datasets used for the ad-NPE recovery plot therefore share identical sequence lengths. As a consequence, the variable-length capability of the ad architecture is not explicitly visible in this evaluation.

3.3.3 Posterior Predictive Check

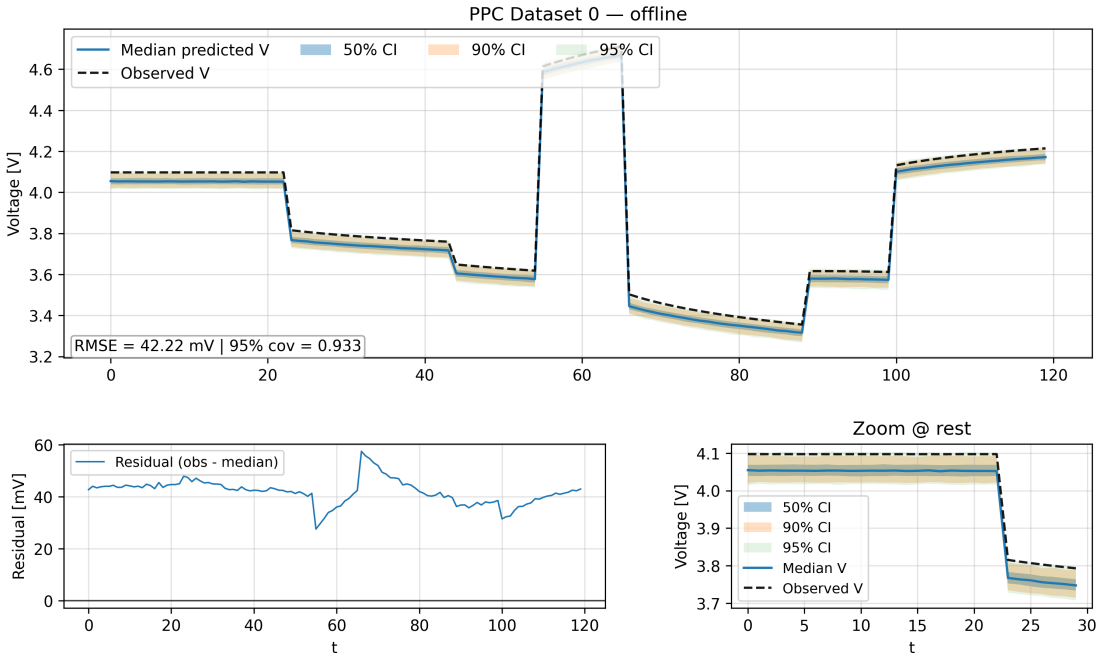


Figure 3.20: Posterior predictive check (PPC) for the synthetic seq-NPE model on a simulated dataset. The observed voltage trace (black) is compared with voltage trajectories simulated from posterior samples. The shaded regions indicate the posterior predictive credible intervals. The residual error and a zoomed-in view are shown at the bottom.

Figure 3.20 shows a posterior predictive check for the seq-NPE model. Starting from a simulated dataset (black curve), posterior samples are inferred and propagated through the forward model to generate new voltage trajectories. The resulting posterior predictive distribution is summarized by the median prediction and credible intervals. The 95% credible interval captures the original voltage trajectory across almost the entire time horizon, indicating that the inferred posterior distribution is consistent with the data-generating process.

This experiment acts as a sanity check for the inference pipeline: data are first generated from the simulator, parameters are inferred from the observations, and new datasets are generated using posterior samples. Consistency between the original observations and the posterior predictive distribution confirms that this round-trip procedure behaves as expected.

3.3.4 Impact of Prior-Context Generation and Representation

We additionally compared the different prior-context design choices introduced in Section 2.5, namely (i) empirical versus synthetic prior context generation and (ii) direct sample-based versus parametric summary representations.

Across calibration and recovery experiments, no substantial or systematic performance differences were observed between these variants. Simulation-based calibration results for the seq-NPE variants (Appendix C, Figure C.1) show broadly similar ECDF difference curves. However, the empirical sample-based and synthetic parametric variants exhibit a mild wave-like deviation from the zero line, indicating slightly overconfident (under-dispersed) posteriors. Posterior contraction analysis (Figure C.2) likewise reveals very similar standardized z-score distributions across variants. Finally, parameter recovery results (Figure C.3) demonstrate nearly identical alignment between posterior medians and ground-truth parameters.

From a practical perspective, the synthetic generation strategy offers a clear advantage, as it eliminates the need to train and calibrate an auxiliary NPE solely for prior-context construction. Moreover, it avoids potential bias propagation from a pretrained model. For these reasons, the synthetic parametric variant is used as the default configuration in the subsequent deployment experiments.

3.3.5 Sequential Inference Under Controlled Deployment Scenarios

We evaluate the models under three controlled deployment scenarios:

- constant charging from low initial SoC
- constant discharging from high initial SoC
- pseudo-random binary sequence (PRBS) excitation

These scenarios were chosen to represent common and practically relevant operating modes. Constant charge and discharge profiles provide simple and controlled conditions with predictable SoC evolution, while PRBS excitation represents a more dynamically varying load profile. Together, they allow evaluation of the models under both structured and more variable operating conditions. Each experiment uses a sliding window of 120 s. The seq-NPE is evaluated in two configurations: with sequential prior propagation and with a uniform prior at each step (denoted by "ablation" in the following). Unless stated otherwise, the sequential model evaluated in this section corresponds to the synthetic training variant using the parametric prior representation $(m, \log(\kappa))$ described in Section 2.5.

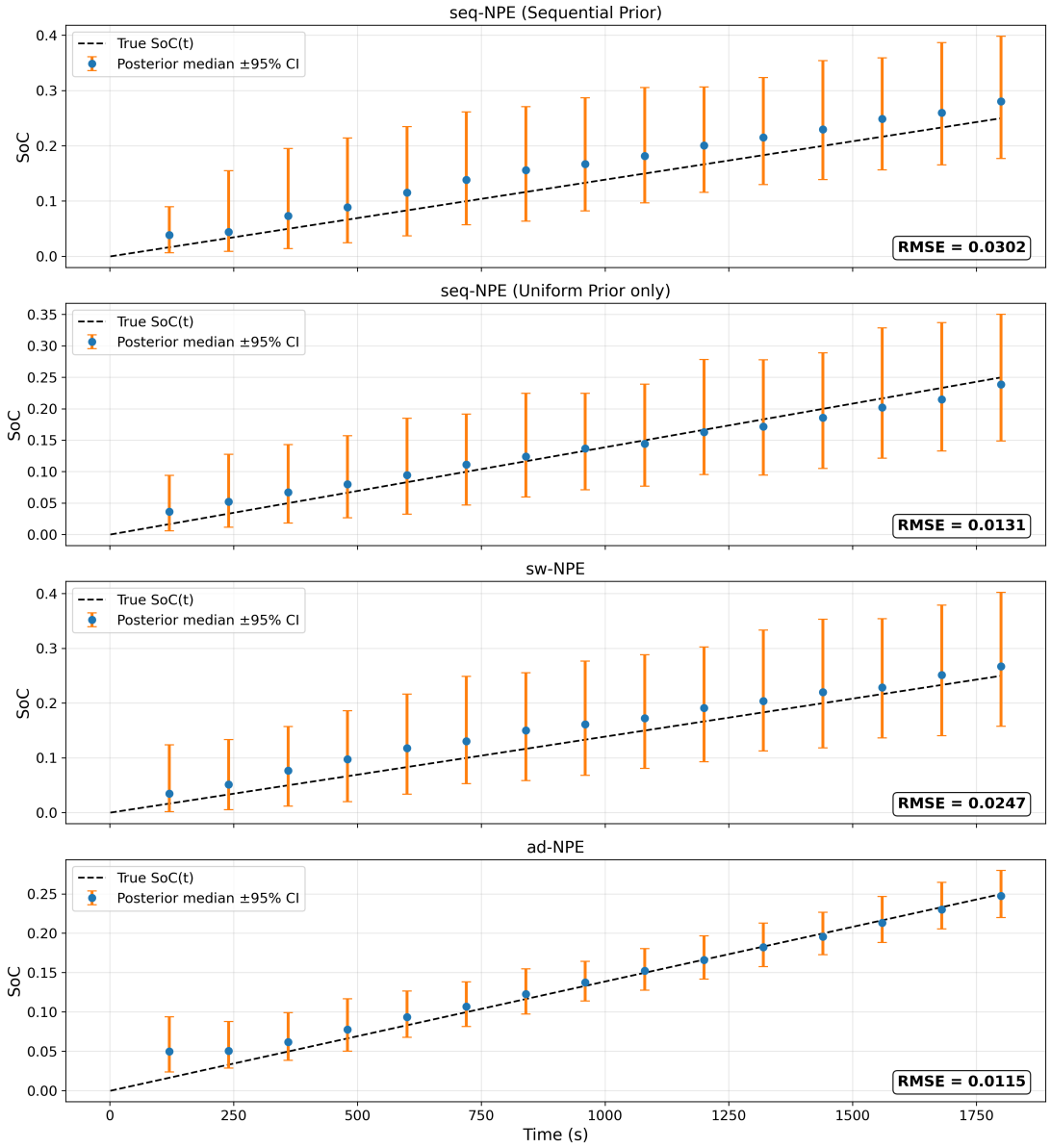


Figure 3.21: Deployment under constant charging from low initial SoC. From top to bottom: seq-NPE with sequential prior propagation, seq-NPE with uniform prior only (ablation), sw-NPE, and ad-NPE. Posterior medians and 95% credible intervals are shown at each 120 s window. The dashed line indicates the ground-truth SoC trajectory.

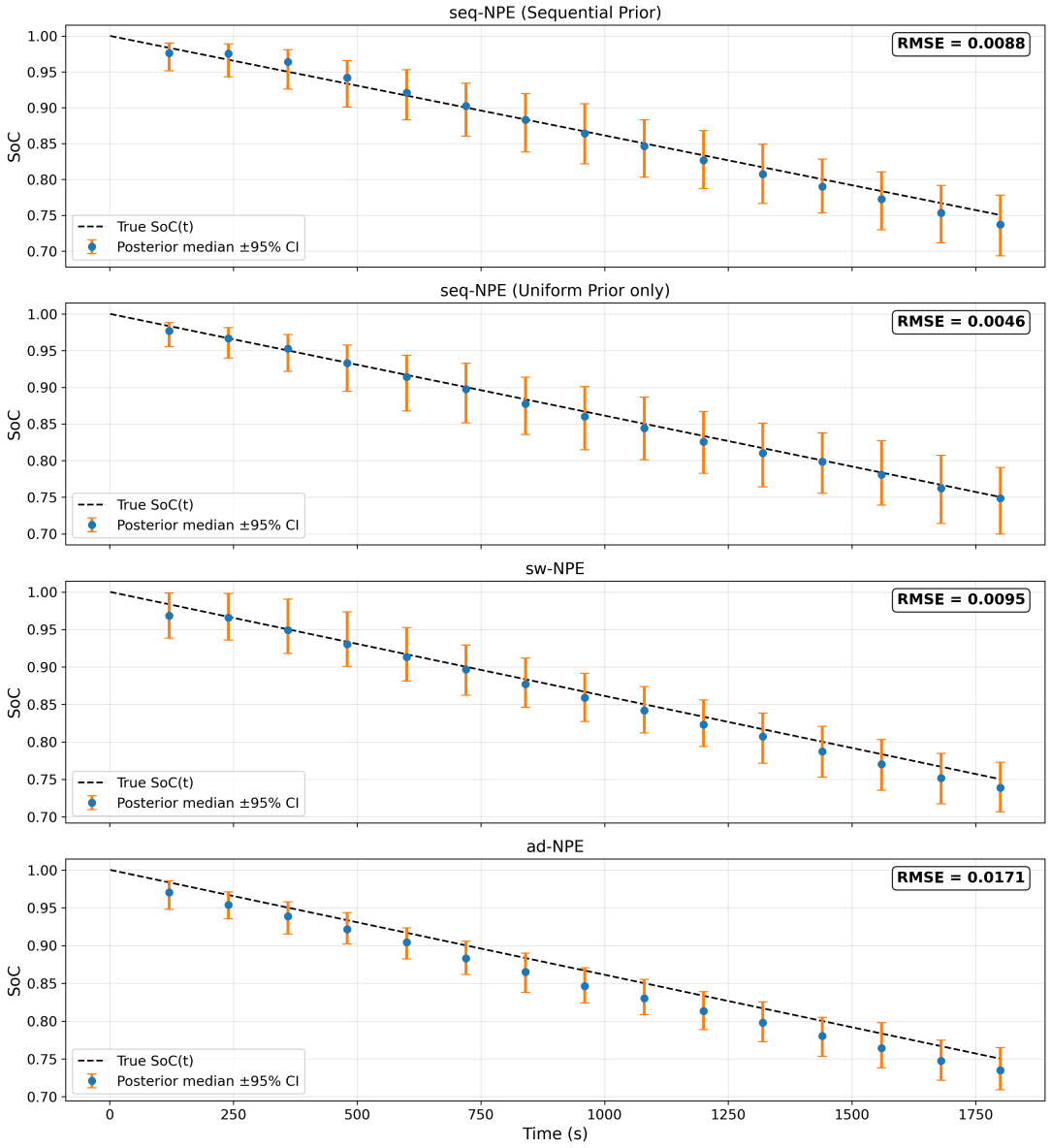


Figure 3.22: Deployment under constant discharging from high initial SoC. Panel ordering is identical to Figure 3.21.

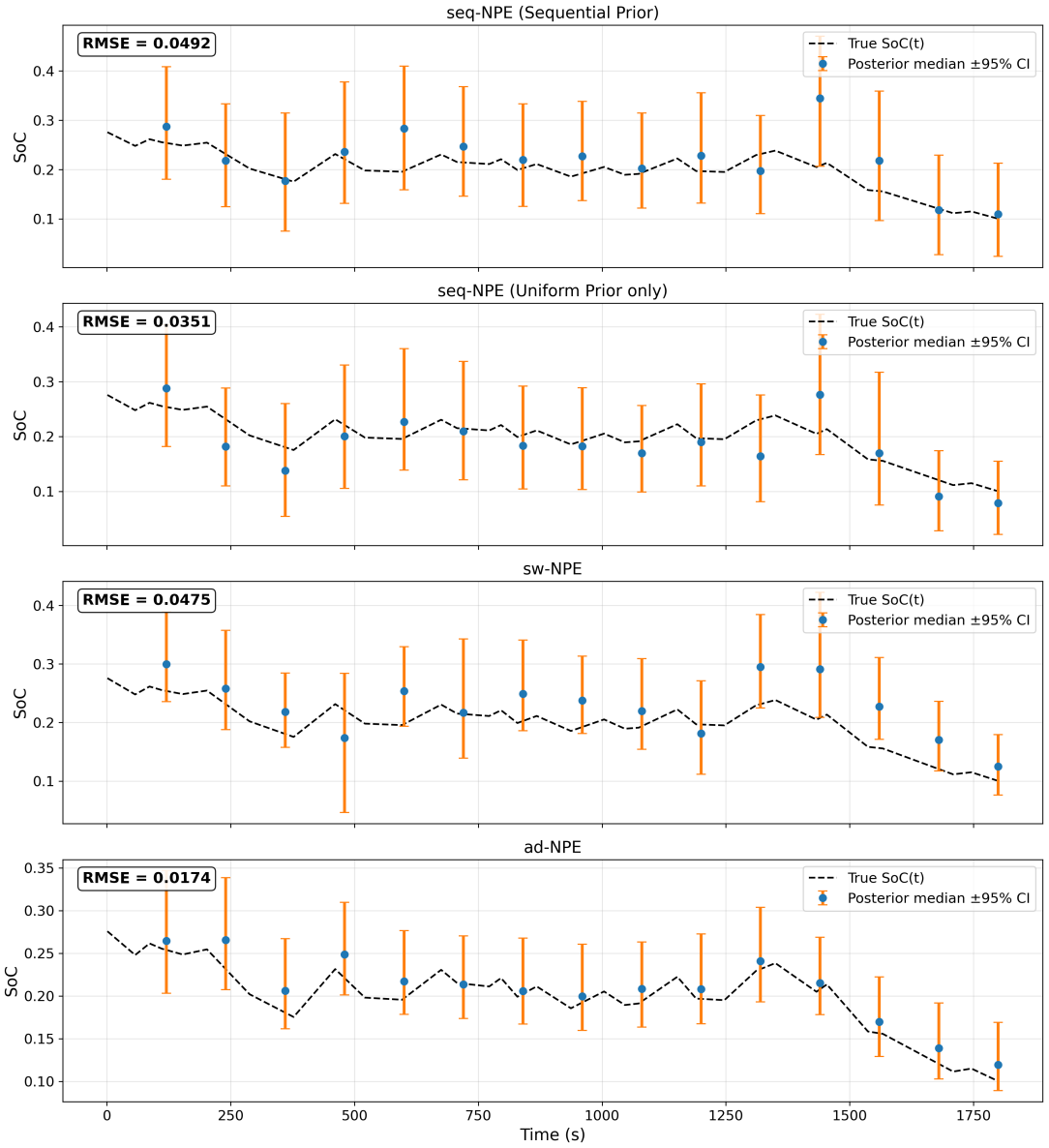
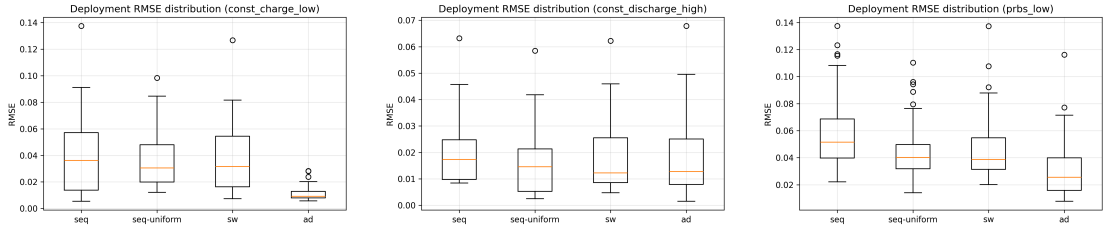


Figure 3.23: Deployment under PRBS load excitation. Panel ordering is identical to Figure 3.21.

Figures 3.21–3.23 show representative deployment trajectories. While sequential prior propagation occasionally stabilizes the posterior trajectory, qualitative improvements over the short-window baseline are not consistently visible across scenarios. To obtain statistically meaningful conclusions, we conducted a multi-seed benchmark with 50 independent random seeds per scenario. We report the full empirical distribution of RMSE values across seeds using boxplots in Figure 3.24.



(a) Const. charge **(b)** Const. discharge **(c)** PRBS excitation

Figure 3.24: Distribution of RMSE across 50 independent deployment runs for three controlled scenarios, shown as boxplots. The box indicates the interquartile range (IQR), the central line denotes the median, whiskers extend to non-outlier extrema, and points represent outliers. Across all regimes, the sequential model with recursive prior propagation does not demonstrate a consistent performance advantage over the short-window baseline.

Across all three scenarios, the all-data model achieves the lowest error, as expected. The sequential model with propagated priors does not improve over the short-window baseline; instead, it typically yields higher RMSE than both sw-NPE and the uniform-prior ablation. The boxplots reveal substantial variability across seeds, with wide interquartile ranges and occasional outliers, indicating noticeable run-to-run variability. Taken together, these results show that, under the investigated conditions, sequential prior propagation does not provide a statistically robust performance gain and is associated with degraded average performance compared to short-window inference.

3.3.6 Distributional Diagnostics (Empirical Model)

The analyses presented in the previous sections focused on the synthetically trained model. In this subsection, we provide additional distributional diagnostics for the empirically trained model. These diagnostics are intended to illustrate internal distributional behavior and prior-posterior dynamics, rather than to provide a direct performance comparison with the synthetic benchmark.

Figure 3.25 compares the (α, β) parameters encountered during training with those inferred during deployment. The deployment samples lie within the support of the training distribution, indicating that inference operates within the regime covered during simulation. No systematic extrapolation beyond the training domain is observed.

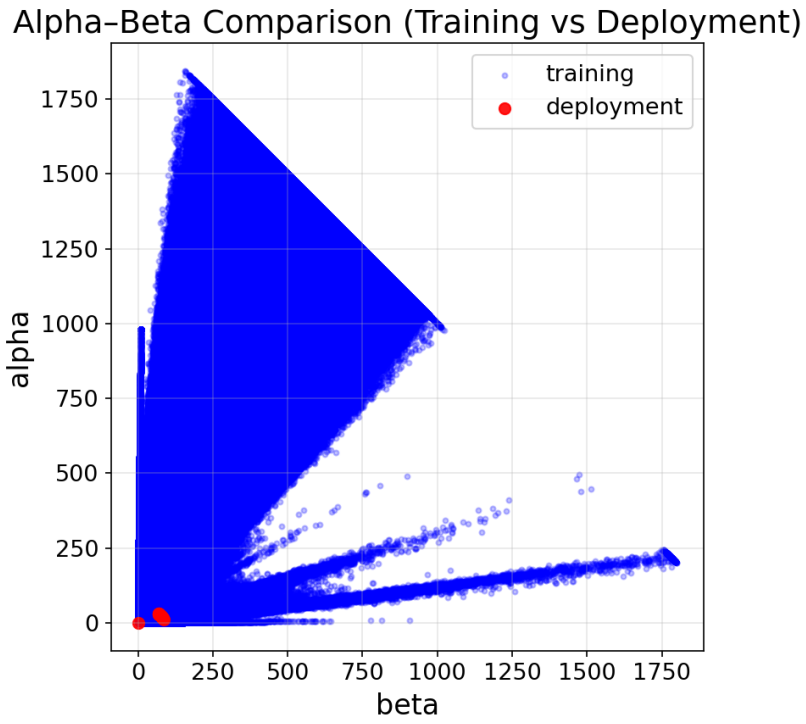


Figure 3.25: Comparison of Beta distribution parameters (α, β) encountered during training (blue) and during deployment (red) for the empirical model. Deployment parameters remain within the support of the training distribution.

The sequential model can represent prior context using a parametric Beta distribution characterized by its mean and log-concentration parameter $\log(\kappa)$. Figure 3.26 illustrates how this parametric representation appears in practice. For consecutive windows, the prior distribution provided to the model and the resulting posterior distribution are shown. The posterior of window t is fitted with a Beta distribution and propagated as the prior for window $t + 1$. We start with a uniform prior context for the first dataset.

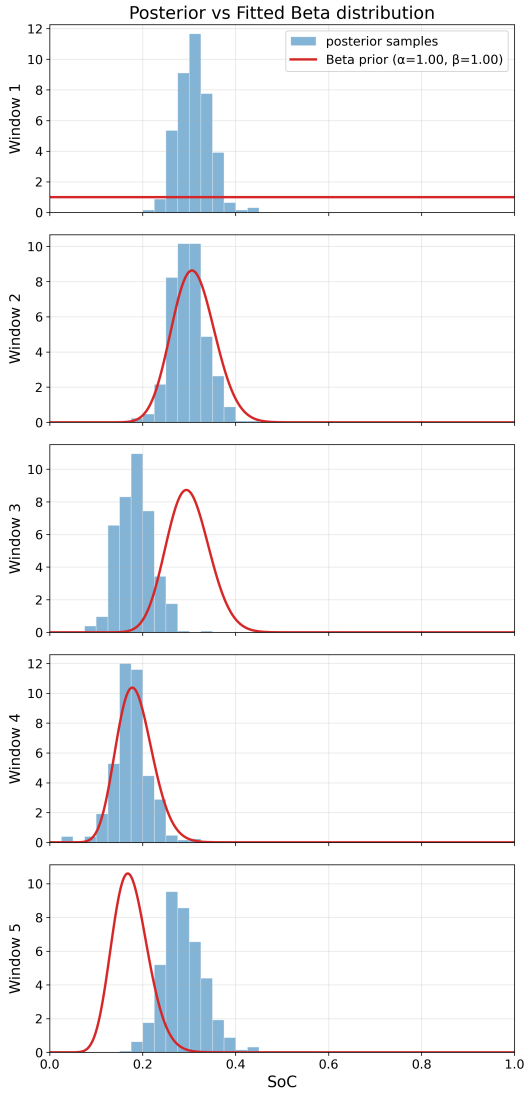


Figure 3.26: Sequential prior-posterior evolution over five consecutive windows for the empirically trained model. The fitted posterior of each window becomes the prior of the next.

3.4 Inference Runtime Comparison: NPE vs SMC-ABC

A central motivation for simulation-based inference with neural posterior estimation (NPE) is its *amortized inference* property, which substantially reduces computational cost at deployment time. Once trained, inference for obtaining a single posterior sample conditioned on a new observation requires only a single forward pass through the neural network. This contrasts with classical likelihood-free methods such as Approximate Bayesian Computation (ABC), which rely on repeated forward simulations during in-

ference. In this section, we quantitatively compare the wall-clock inference time of the trained NPE model against Sequential Monte Carlo ABC (SMC-ABC) for SoC estimation. Both methods are evaluated on the same dataset and under identical computational conditions.

3.4.1 Experimental Setup

All runtime measurements were performed on a shared high-performance computing (HPC) system using CPU-only execution. Jobs were executed on the ETH Zurich Euler cluster, with each run restricted to a single physical CPU core to ensure fair and reproducible timing. Specifically, computations were carried out on AMD EPYC 7763 processors (64-core, Zen 3 architecture, base frequency 2.45 GHz), with one core allocated per task and 8 GB of memory. Implicit thread-level parallelism was disabled to avoid oversubscription and ensure reproducible wall-clock timing. For both inference methods, $N = 2000$ posterior samples were generated for a single fixed observation window of length 600 s sampled at 10 Hz. The following runtime quantities were measured:

- NPE inference time for generating N posterior samples,
- wall-clock time for N standalone ECM forward simulations, and
- total SMC-ABC inference time required to obtain N accepted samples.

Neural posterior inference was performed strictly in inference mode, without any additional forward simulations or optimization steps. All experiments were repeated multiple times using identical code and configuration to assess run-to-run variability.

3.4.2 Runtime Results

SMC-ABC inference was carried out using the `pyabc` [17] framework, employing the same 3-RC equivalent circuit model used during NPE training as the forward simulator. The ABC prior included uncertainty in both the terminal state of charge SoC and the cell capacity Q_{eff}, with additional stochastic perturbations of ECM parameters to match the NPE training setup. A scalar root-mean-square error (RMSE) between simulated and observed terminal voltage trajectories was used as summary statistic. SMC-ABC was run with a population size of 2000 particles and an adaptive median-based acceptance threshold. Across all runs, the algorithm required 9867 forward simulations to obtain the final posterior population (same seed used across runs). This corresponds to an average acceptance rate of approximately 20% in the final population. Table 3.1 summarizes the measured runtimes for both inference approaches. Across all runs, neural posterior inference required approximately 2.7–3.1 s to generate $N = 2000$ posterior samples. In contrast, SMC-ABC required several hundred seconds under identical computational conditions.

Overall, amortized NPE inference achieved a speedup of more than two orders of magnitude relative to SMC-ABC. This performance gap is primarily attributable to the repeated forward simulations required by ABC-based methods, whereas NPE performs

inference directly in parameter space once training has been completed. Minor variability in absolute runtimes across runs is attributed to system-level effects inherent to shared computing environments, such as CPU frequency scaling and operating system scheduling. These variations are negligible compared to the observed difference in computational cost between both methods.

Table 3.1: Runtime comparison between amortized neural posterior estimation (NPE) and Sequential Monte Carlo ABC (SMC-ABC) for $N = 2000$ posterior samples on a single CPU core.

Metric	Run 1	Run 2	Run 3	Run 4	Run 5
NPE inference time [s]	3.05	2.86	2.75	2.71	2.83
ECM forward simulation time for N runs [s]	164.24	155.77	154.17	153.99	153.29
SMC-ABC inference time [s]	833	801	769	779	763
Speedup (ABC / NPE) [×]	273	280	280	288	269

Figure 3.27 compares the posterior distributions obtained with NPE and SMC-ABC. Both methods concentrate posterior mass near the ground-truth terminal state of charge. The posterior medians are in close agreement, indicating that the substantial speedup achieved by NPE does not come at the cost of degraded inference quality.

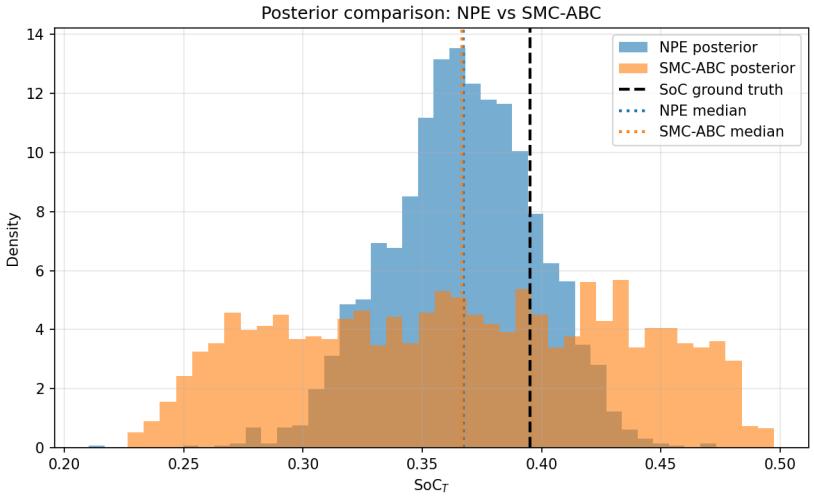


Figure 3.27: Posterior distributions for the terminal state of charge SoC_T obtained with neural posterior estimation (NPE) and SMC-ABC. The dashed vertical line indicates the ground-truth value, while dotted lines denote the respective posterior medians.

Overall, these results empirically confirm the key advantage of neural posterior estimation: once trained, inference can be performed orders of magnitude faster than classical simulation-based methods, making NPE particularly attractive for real-time or large-scale deployment scenarios. It is worth noting that the computational gap could in principle be reduced by replacing the physics-based forward model with a learned

surrogate to accelerate individual simulations. However, it is unclear whether such an approach would sufficiently compensate for the fundamentally larger number of simulations required by SMC-ABC. Moreover, given the relatively simple ODE structure of the employed battery model, it is not evident that a surrogate model would yield a meaningful speed advantage over the existing simulator.

Chapter 4

Discussion

4.1 Inherent Identifiability of SoC and SoH

The joint SoC-SoH experiments reveal a fundamental asymmetry in identifiability between the two latent states. While SoC is recovered with high accuracy and tight posterior concentration across the full state space, SoH exhibits substantially larger posterior dispersion and stronger sensitivity to excitation conditions. This behavior is not primarily a limitation of the neural posterior estimator, but rather reflects a structural property of the underlying physics. Capacity enters the system dynamics indirectly through accumulated charge, such that SoH becomes observable only when sufficient charge throughput occurs within the observation window. The clear relationship between posterior contraction and the total SoC excursion ΔSoC confirms this mechanism. From a practical standpoint, this dependency is not necessarily a disadvantage. Capacity degradation evolves on long time scales, whereas SoC varies rapidly during normal operation. Inferring both quantities jointly within short windows therefore conflates latent variables that evolve on fundamentally different temporal scales. A more consistent strategy is to train models on long time-series inputs when SoH is the primary quantity of interest, allowing sufficient cumulative charge throughput to render capacity identifiable. In contrast, short-window inference is well suited for SoC, which responds immediately to local excitation. The experiments therefore suggest that SoC and SoH inference are best addressed on time scales aligned with their underlying physical dynamics.

4.2 Generalization from Simulation to Experimental ABB Data

A central question of this work is whether a neural posterior estimator trained exclusively on synthetic ECM simulations can generalize to real measurements. The ABB experiments demonstrate that this is possible: despite being trained exclusively on simulated trajectories, the NPE produces accurate and stable SoC estimates on experimental data. This indicates that the learned inverse mapping captures the essential physical struc-

ture encoded in the simulator. The robustness experiments provide further insight into the generalization properties. First, performance remains stable when training currents are generated using simplified procedural or sinusoidal profiles, as long as these profiles contain sufficient dynamic excitation. Only degenerate constant currents lead to substantial degradation, which is consistent with the model not being exposed to dynamic load behavior during training. This suggests that exact replication of real current statistics is not necessary; rather, structural richness in excitation is the key requirement for learning an inverse mapping that generalizes. Second, applying a model trained at 25 °C to data recorded at 45 °C leads to localized performance degradation, particularly in SoC regions where the OCV-SoC curves differ most strongly between temperatures. Importantly, the model does not fail arbitrarily but degrades in a physically interpretable manner, indicating that the dominant mismatch arises from systematic shifts in the OCV mapping.

Overall, these results support the view that simulator-based training can be viable for real data, provided that the simulator captures the dominant physical mechanisms. The observed robustness to moderate model misspecification, together with the inclusion of parameter variability during training to account for cell-to-cell differences, suggests a potential extension of the approach beyond a single calibrated battery model. In particular, one could hypothesize that incorporating a broader range of OCV-SoC relationships during training might enable a single NPE model to generalize across different batteries with slightly varying characteristics. While this has not been explicitly evaluated in this work, the present results indicate that such an extension may be feasible.

Beyond this, the likelihood-free nature of the approach provides additional flexibility with respect to the underlying forward model. While this study employs an equivalent circuit model (ECM), such models are known to have limitations in capturing all relevant battery dynamics. The simulation-based inference framework is not tied to a specific model class and allows the simulator to be replaced by more detailed or physically accurate models if available. This decoupling between inference and model choice suggests that improved forward models could be incorporated without fundamentally changing the inference methodology.

4.3 Sequential Posterior Propagation

The sequential NPE (seq-NPE) was introduced to propagate information between consecutive short windows and thereby improve inference relative to a fixed short-window baseline (sw-NPE). Empirically, the results do not demonstrate a consistent improvement of seq-NPE over sw-NPE. While occasional gains are observed in the controlled deployment scenarios, the sequential model does not reliably outperform the short-window baseline across regimes. Three observations are central to interpreting this outcome.

First, posterior contraction for 120 s windows is already very high across models, indicating that short windows in the considered regimes are highly informative. When a single window already provides strong identifiability, the potential benefit of additional prior propagation is inherently limited, and measurable improvements become

structurally difficult to obtain.

Second, sequential inference requires the amortized model to implicitly learn a gating mechanism: it must detect whether the propagated prior context and the current data are consistent. When consistent, the posterior should tighten; when inconsistent, the model should ignore the prior context and rely primarily on the data. The observed variability suggests that the architecture struggles to robustly differentiate between these cases, resulting in situations where the prior context is under-weighted and others where it is overly influential.

Third, the sequential formulation only propagates a compressed representation of past information through the prior context, rather than retaining the full measurement history. In contrast, the ad-NPE has access to the complete trajectory and therefore represents an approximate upper bound on the achievable performance. The fact that seq-NPE does not consistently match this upper bound suggests that the chosen prior representation does not fully capture all relevant information from previous windows, indicating a potential loss of information during propagation.

This does not invalidate the sequential idea, but it highlights that combining prior context and current data in a robust way is challenging in practice. In the present setting, propagating SoC offers limited benefit because the short-window inference problem is already well-posed. When a single window provides strong identifiability, the potential gain from sequential prior propagation is small. In contrast, sequential propagation may be more promising for latent quantities that are only weakly identifiable from short windows and whose information accumulates over longer time horizons. This perspective motivates the alternative propagation strategies discussed in Section 4.3.1.

4.3.1 Sequential Prior Propagation for Weakly Identifiable Parameters

Motivated by the observation that sequential propagation may be more beneficial for weakly identifiable quantities, an additional exploratory experiment was conducted in which the propagated prior context was defined in terms of SoH rather than SoC. The goal was to assess whether sequential prior propagation can help stabilize inference for latent variables whose information accumulates only gradually over time. In this setting, the short-window model and a sequential SoH-propagating variant were compared using 120 s windows. Representative examples are shown in Figure 4.1 and Figure 4.2.

Figure 4.1: Preliminary experiment on sequential SoH propagation. In this example, propagating a structured SoH prior context across windows leads to accurate and stable tracking of the ground-truth SoH. The sequential model remains closely aligned with the true trajectory, while the short-window baseline exhibits larger deviations covering the prior space. The ablation (middle panel) confirms that convergence is driven by the propagation of the prior. The dashed line indicates the ground-truth SoH.

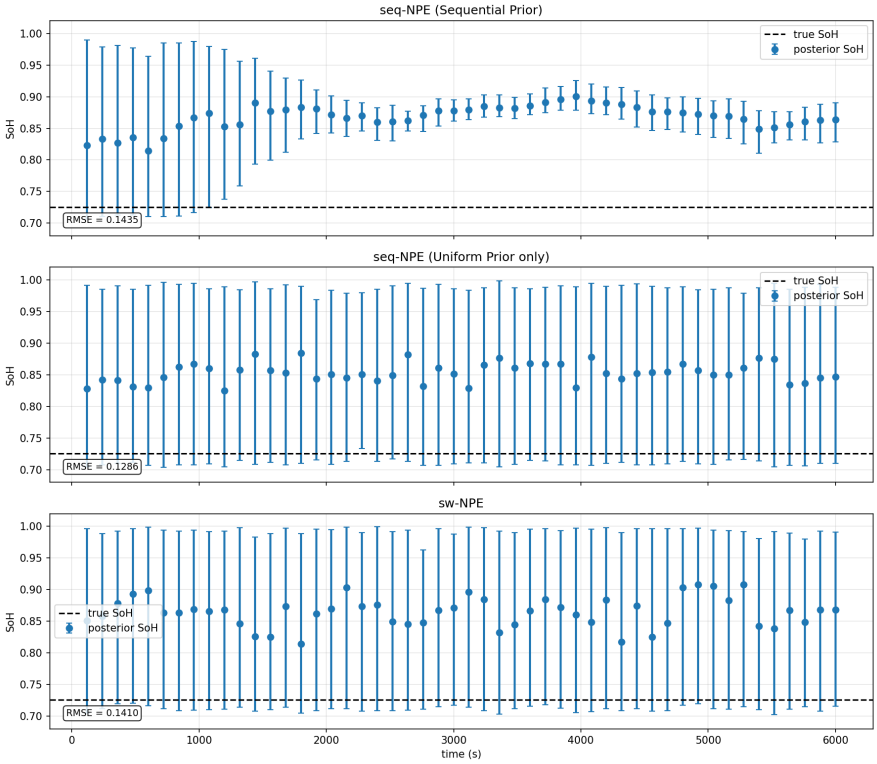


Figure 4.2: The sequential model exhibits posterior contraction but converges to an incorrect value, missing the ground truth by more than 0.1. In contrast, the short-window model remains diffuse and largely reflects the prior range for SoH, indicating that little information about SoH is contained in the individual short segments.

The results illustrate both the potential and the limitations of this approach. In certain cases (Figure 4.1), sequential propagation of SoH leads to progressive posterior contraction towards the correct value, resulting in stable tracking of the ground-truth trajectory. In other cases (Figure 4.2), the sequential model similarly exhibits posterior contraction but converges towards an incorrect value. In contrast, the short-window baseline remains largely uninformative in both scenarios, with broad uncertainty spanning most of the feasible SoH range.

This experiment was conducted at the end of the study, and no dedicated architectural modifications or extensive hyperparameter optimization were performed for the SoH propagation setting. The results should therefore be interpreted as a preliminary indication rather than a fully optimized comparison. Nevertheless, the experiment highlights a conceptual distinction: sequential amortized inference may be particularly relevant for parameters whose information accumulates gradually over time, such as SoH, internal resistance drift, or stochastic noise parameters. In such settings, propagating structured posterior summaries across windows could help compensate for the limited information content of individual short segments.

4.4 Computational Efficiency and Practical Implications

The runtime comparison against SMC-ABC highlights the central advantage of neural posterior estimation: amortization inference. Both methods were evaluated under identical hardware conditions and using the same forward model. While SMC-ABC required several hundred seconds to obtain $N = 2000$ posterior samples, the trained NPE produced an equally sized posterior sample in approximately three seconds. This corresponds to a speedup of more than two orders of magnitude. Importantly, this acceleration does not come at the expense of inference quality. The posterior distributions obtained with NPE closely match those produced by SMC-ABC, with similar concentration around the ground-truth terminal SoC and comparable posterior medians. The gain is therefore not a trade-off between speed and accuracy, but rather a redistribution of computational effort from inference time to training time. In practical terms, this means that once a neural posterior estimator has been trained, repeated Bayesian inference queries can be answered almost instantaneously. This property is particularly relevant in settings where frequent state updates are required, such as online battery management or large-scale evaluation across many trajectories. The results therefore demonstrate that amortized simulation-based inference can retain the statistical advantages of Bayesian methods while dramatically reducing deployment-time computational cost.

Chapter 5

Conclusions and Outlook

5.1 Conclusions

This thesis investigated the application of neural posterior estimators (NPEs) to the problem of battery state estimation. The objective was to infer the state of charge (SoC) and state of health (SoH) from current and voltage measurements, using an equivalent-circuit model (ECM) for training.

First, the initial experiments demonstrate that SoC inference is substantially easier than SoH inference. This behavior is expected, since SoH becomes clearly observable only through sufficiently large changes in SoC. Consequently, for reliable SoH inference, it is advisable to train models on long time-series data (e.g., days or weeks of operation) that contain substantial ΔSoC. This recommendation is also consistent with practical deployment considerations, since SoH evolves on the time scale of weeks, months, or years. Under such conditions, the requirement for long data horizons does not constitute a limitation of the approach.

Second, the study demonstrates that an NPE trained exclusively on synthetic ECM simulations can generalize to real battery data. Accurate SoC inference was achieved without direct training on measured trajectories, thereby demonstrating the generalization capability of the method. The model remains robust under moderate current-model misspecification and exhibits physically interpretable degradation under variations in the underlying battery characteristics, for example changes in the OCV-SoC relationship. These findings indicate that simulator-based training is viable when the forward model captures the dominant physical mechanisms of the system.

Third, a sequential neural posterior estimator (seq-NPE) was investigated with the objective of enabling inference on short measurement windows while propagating information between consecutive segments. Multiple prior representations and generation strategies were evaluated to assess whether sequential context improves estimation accuracy relative to a fixed short-window baseline. Across all tested deployment scenarios, the sequential variants performed comparably to or slightly worse than the short-window model and did not demonstrate a consistent improvement. No configuration could be identified that reliably outperformed the non-sequential short-window approach. Within

the scope of this study, a clear performance advantage of the sequential approach over the short-window baseline could therefore not be established.

Finally, a direct runtime comparison with SMC-ABC demonstrates that NPE achieves speedups exceeding two orders of magnitude while maintaining comparable posterior quality. The computational gain is not achieved through approximation at inference time, but through amortization: computational effort is shifted to the training phase, enabling near-instantaneous posterior sampling once the model has been trained.

Taken together, the results demonstrate that neural posterior estimators can be effectively applied to battery state estimation when trained on physically motivated ECM simulators. In particular, the study shows that an NPE can generalize to real experimental battery data for SoC estimation without requiring direct training on measured trajectories. Furthermore, comparable posterior quality to SMC-ABC can be achieved at a computational cost that is more than two orders of magnitude lower at inference time.

5.2 Outlook

Three directions appear particularly promising for extending the present work.

First, experimental validation should be extended to batteries undergoing substantial long-term degradation. In this thesis, experimental data were limited to cells operating close to nominal capacity, so that SoH inference could not be assessed under realistic capacity-fade conditions. A natural continuation would therefore be to analyze batteries with measurable degradation over extended periods. Such data would allow a more rigorous evaluation of whether a neural posterior estimator trained on simulated degradation scenarios can reliably track SoH evolution in practice.

Second, future work should investigate prior adaptation for weakly identifiable latent variables. As discussed in Section 4.3.1, the sequential experiments in this thesis focused primarily on propagating SoC-related posterior information between short windows. Since short windows were already highly informative for SoC, sequential prior propagation offered limited potential for measurable improvement. In contrast, latent quantities such as SoH become observable only through sufficiently large cumulative charge throughput and are therefore much more weakly identifiable from short segments. This suggests that sequential amortized inference may be more beneficial for slowly varying or weakly identifiable variables, such as SoH, internal resistance drift, or stochastic noise parameters, where propagating structured posterior summaries across windows could compensate for the limited information content of individual observations.

A related continuation concerns alternative mechanisms for prior adaptation in amortized inference models. Recently, a diffusion-based method for test-time prior adaptation was proposed in [11], allowing the prior distribution to be adapted after training without retraining the model. Although developed in a diffusion framework, the underlying objective is closely related to the sequential approach considered here, namely incorporating updated prior information during deployment. A direct comparison between sequential posterior propagation and explicit test-time prior adaptation would therefore

represent a promising avenue for future research.

Third, the approach should be extended to more realistic battery models. While this thesis employs an equivalent-circuit model (ECM), more detailed electrochemical models are known to capture a broader range of battery dynamics but are also significantly more computationally demanding. In such settings, the benefits of amortized inference are expected to become even more pronounced, as the high cost of repeated forward simulations would make classical simulation-based methods increasingly impractical. Extending neural posterior estimation to these more complex models therefore represents a promising direction for future work.

Appendices

Appendix A

Robust Parameter Fitting and Jitter Sampling

To model realistic cell-to-cell variability, each parameter series $y_i \in \{R_0, R_1, R_2, C_1, C_2\}$, given as a set of discrete values corresponding to specific states of charge (SoC), is fitted in log-space using a robust outlier rejection scheme. This procedure ensures positivity, reduces sensitivity to extreme points, and provides a stable estimate of the mean and spread for subsequent jitter sampling.

Iterative MAD-based Outlier Rejection

Given discrete state-of-charge points x_i and corresponding positive parameter values y_i, each parameter series is fitted in log-space to ensure positivity and to capture multiplicative trends.

1. **Log transform.** Transform the parameter values but not the SoC grid:

$$\ell_i = \log(y_i).$$

2. **Linear fit.** Fit a first-degree polynomial $\ell_i \approx a + b\,x_i$ to the current inlier points (initially all data).

3. **Compute log-residuals.** The fitted values are $\widehat{\ell_i} = a + b\,x_i$, and the residuals are

$$r_i = \ell_i - \widehat{\ell_i}.$$

4. **Estimate robust scale using the Median Absolute Deviation (MAD).**

$$\sigma_{\text{MAD}} = 1.4826\,\text{median}\big(|r_i - \text{median}(r)|\big).$$

   The constant 1.4826 rescales the MAD so that it is comparable to the standard deviation under a normal distribution (MAD $\approx 0.6745\,\sigma$ for Gaussian data).

5. **Outlier rejection.** Points with $|r_i - \text{median}(r)| > k\sigma_{\text{MAD}}$ (with $k = 2.5$ in our implementation) are treated as outliers. The model is then refitted on the remaining inliers.

The final inliers define a smooth mean curve $f(\text{SoC})$ in log-space, which is exponentiated back to obtain the fitted parameter values in SI units. In addition, a global scaling factor σ_{scale} is applied to the estimated log-space standard deviations of all parameters to control the overall degree of variability among simulated cells. During training, this factor can be scheduled progressively, starting from smaller values to present the model with low-variability (easier) examples and gradually increasing it to introduce more realistic cell-to-cell differences. This approach improves training stability and encourages the network to generalize across a broader range of parameter variations.

Estimation of Natural Variability

After outlier removal, the residuals of the inlier points quantify the natural scatter of each parameter around its fitted mean curve. Their dispersion in log-space provides an estimate of the multiplicative variability observed in the data. When generating synthetic cells, this spread defines the width of a log-normal distribution from which multiplicative jitter factors are sampled, ensuring that all simulated parameters remain positive and vary proportionally to their mean trends.

The variations of R_i and C_i within each RC branch are sampled in a correlated manner. This means that when R_i decreases, C_i tends to increase, and vice versa, so that their product $\tau_i = R_i C_i$ (the time constant) remains within a realistic range. The correlation between these parameters is estimated directly from the observed variability in the data, ensuring that the simulated cells reflect the same coupling seen in the fitted parameter curves.

OCV Bias

The OCV curve was interpolated smoothly between tabulated SoC points to obtain a continuous function $OCV(\text{SoC})$. To simulate small cell-to-cell deviations, an independent constant voltage bias

$$V_{\text{bias}} \sim \mathcal{N}(0,\, 5\,\text{mV})$$

is applied once per simulated trace. This models small systematic offsets in the open-circuit voltage characteristic while preserving the overall shape of the OCV curve.

Alternative OCV variability model. In preliminary experiments for the joint SoC-SoH estimation model (Section 3.1), a more flexible OCV perturbation scheme was investigated. Instead of applying a constant voltage offset, correlated perturbations were sampled at the tabulated SoC support points of the OCV curve and then interpolated using a shape-preserving spline. This produces smooth cell-to-cell variations in the shape of the OCV characteristic rather than a simple vertical shift.

The motivation for exploring this approach was that battery aging and environmental factors such as temperature may alter the OCV curve in more complex ways than a constant bias. However, the exact mechanisms governing OCV shape changes remain an active area of research. For the majority of models considered in this thesis, the simpler constant-bias formulation was retained because it provides a stable approximation while keeping the simulator well constrained.

Illustration of Parameter Variability

To illustrate the effect of the introduced parameter variability, Figures A.1–A.4 show representative parameter curves for the open-circuit voltage (OCV), resistances, time constants, and capacitances as functions of SoC.

For visualization purposes, the OCV bias may be slightly exaggerated in the plots. In the actual simulations, a constant OCV bias with a standard deviation of 5 mV was used, resulting in only minor shifts on the full voltage scale.

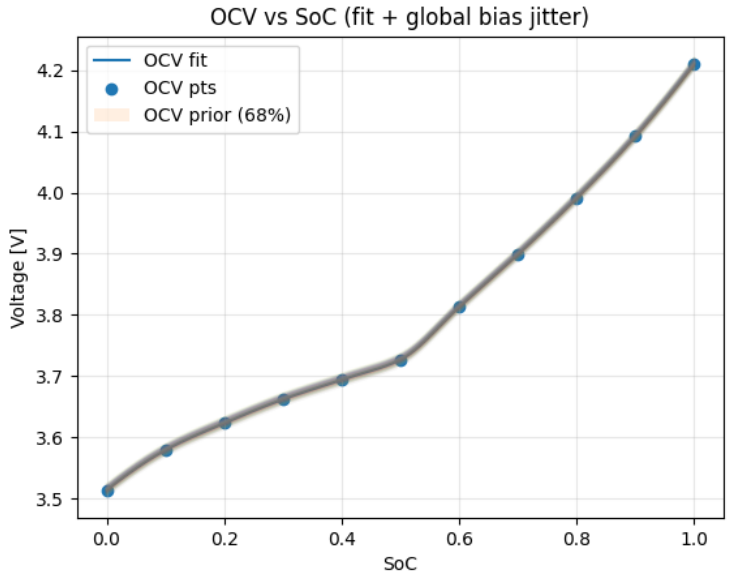


Figure A.1: OCV as a function of SoC with random constant bias applied to each simulated trace.

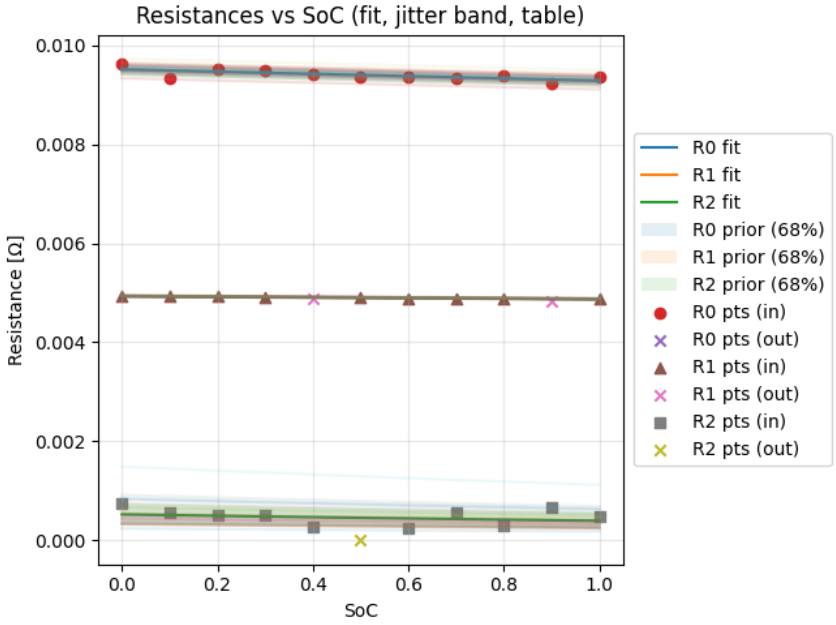


Figure A.2: Resistances (R_0, R_1, R_2) as a function of SoC. The points represent tabulated parameter values, with symbols indicating inliers and outliers based on the MAD-based outlier rejection.

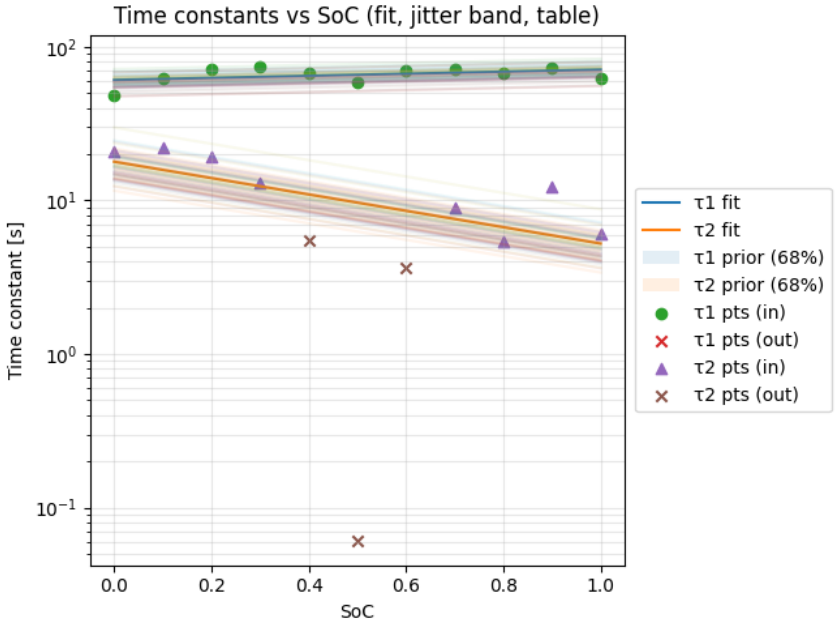


Figure A.3: Time constants τ_1 and τ_2 as a function of SoC, derived from the corresponding R_iC_i products.

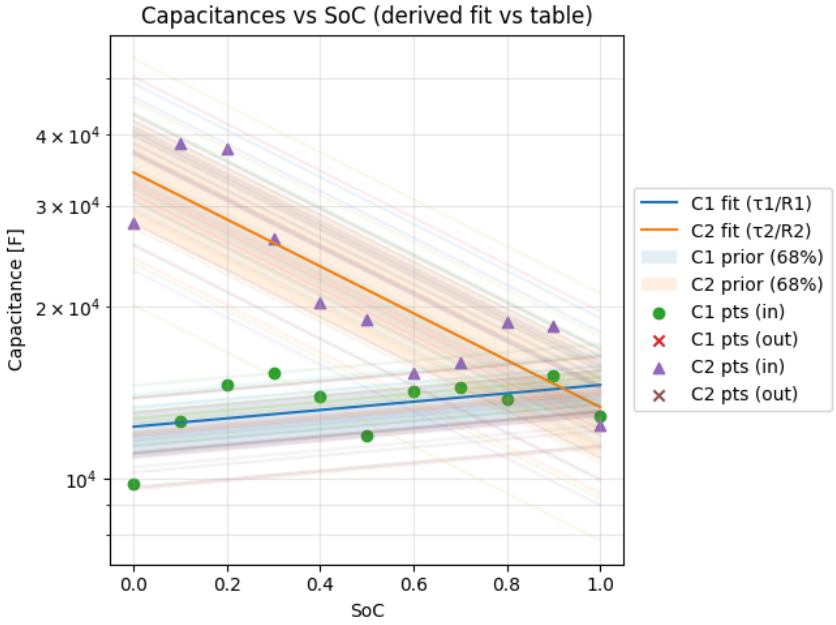


Figure A.4: Capacitances (C_1, C_2) as a function of SoC, derived from τ_i/R_i.

Appendix B

Algorithms

B.1 Algorithmic summaries of current generation

Algorithm 1 Sampling a current trajectory from a precomputed KLE model

Input: Mean current $\mu_I(t)$, orthonormal eigenvectors $\{\phi_n(t)\}_{n=1}^{K}$ of the empirical covariance matrix, empirical coefficient matrix $\mathbf{A} = \mathbf{US}$

Output: Current trajectory $I(t)$

1: Sample coefficient vector $\boldsymbol{\xi} \leftarrow$ random row of $\mathbf{A}$
2: $I(t) \leftarrow \mu_I(t) + \sum_{n=1}^{K} \xi_n \phi_n(t)$
3: **return** $I(t)$

Algorithm 2 Simplified synthetic current generation

Input: Target duration T, sampling interval Δt

Output: Current trajectory $I(t)$

$m \in \{\text{rest}, \text{constant}, \text{step}, \text{PRBS (pseudo-random binary sequence)}, \text{mixed}\}$

1: Generate piecewise-constant current $I(t)$ according to mode m
2: **return** $I(t)$

Algorithm 3 Sinusoidal current generation

Input: Target duration T, sampling interval Δt

Output: Current trajectory $I(t)$

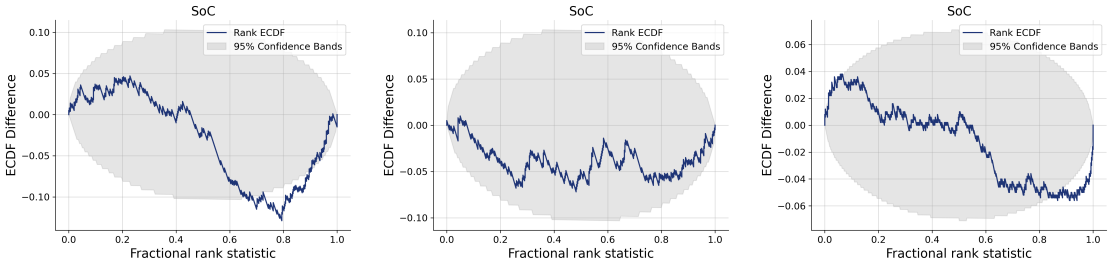
1: Sample frequency $f \sim \mathcal{U}(f_{\min}, f_{\max})$, amplitude $A \sim \mathcal{U}(0, A_{\max})$, and phase $\phi \sim \mathcal{U}(0, 2\pi)$
2: Generate sinusoidal current:
3: $\quad I(t) \leftarrow A \sin(2\pi f t + \phi)$
4: **return** $I(t)$

Algorithm 4 Degenerate constant current generation

Input: Target duration T, sampling interval Δt
Output: Current trajectory $I(t)$
1: Set $I(t) \leftarrow I_0$ for all $t \in [0, T]$, with fixed magnitude $I_0 = 0.02\,\mathrm{A}$
2: **return** $I(t)$

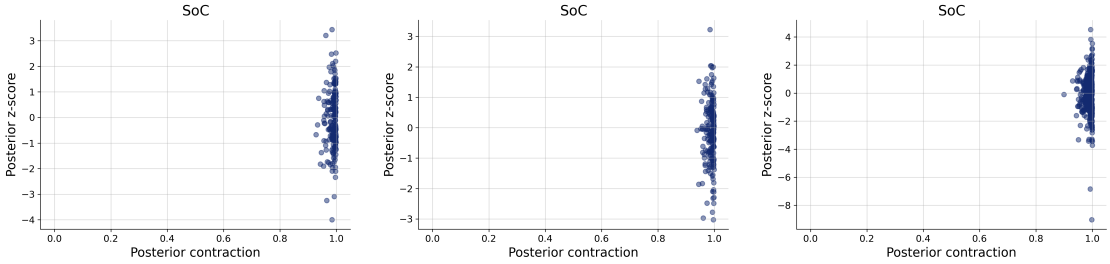
Appendix C

Additional Calibration Results for seq-NPE Variants



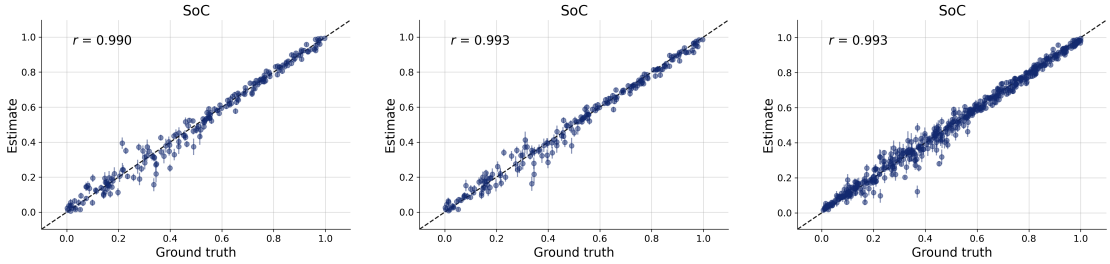
(a) Empirical (samples) **(b)** Empirical ($[\mu, \log(\kappa)]$) **(c)** Synthetic ($[\mu, \log(\kappa)]$)

Figure C.1: Simulation-based calibration (SBC) ECDF difference plots for the three seq-NPE variants. The empirical variants differ in prior representation (sample-based vs. parametric), while the synthetic variant uses the parametric prior representation. The empirical parametric variant appears well calibrated, while the other two variants show a mild tendency toward over-confident posteriors.



(a) Empirical (samples) **(b)** Empirical ($[\mu, \log(\kappa)_{\text{norm}}]$) **(c)** Synthetic ($[\mu, \log(\kappa)_{\text{norm}}]$)

Figure C.2: Posterior contraction analysis for the seq-NPE variants. Standardized z-scores are computed as $(\theta_{\text{true}} - \mu_{\text{post}})/\sigma_{\text{post}}$. All variants show similar contraction behavior, indicating comparable uncertainty calibration.



(a) Empirical (samples) **(b)** Empirical ($[\mu, \log(\kappa)_{\text{norm}}]$) **(c)** Synthetic ($[\mu, \log(\kappa)_{\text{norm}}]$)

Figure C.3: Parameter recovery comparison for the seq-NPE variants. All three variants show similar recovery behavior, indicating that neither prior representation nor prior generation strategy substantially alters predictive accuracy.

Appendix D

Implementation in BayesFlow

All neural posterior estimators in this thesis were implemented using the *BayesFlow* framework [7], which provides a modular architecture for simulation-based inference. The implementation follows BayesFlow's separation into *simulator*, *adapter* and *workflow* components.

Simulator

The simulator represents the generative model and is composed of three functions: `meta()`, `prior()`, and `likelihood()`. The `meta()` function samples auxiliary simulation settings (e.g. sequence lengths) that control structural aspects of the simulation. The `prior()` function samples the latent parameters θ (e.g. SoC, SoH, noise scales, and bias terms) according to the prior distributions defined in Section 2.3. The `likelihood()` function implements the forward model, including generation of synthetic current profiles, simulation of the ECM, addition of measurement noise, and formatting of the resulting time-series outputs. The simulator is wrapped using BayesFlow's `make_simulator` interface, enabling online simulation during training.

Adapter

The adapter is the component responsible for transforming simulator outputs into the structured inputs required by the machine learning pipeline. It defines how raw simulation data are selected, constrained, and organized before being processed by the neural networks.

In the present work, the adapter is used to:

- specify which simulator outputs are retained for training and diagnostics,
- enforce physically meaningful and numerically stable parameter ranges,
- define the summary variables (conditioning inputs) and inference variables (target parameters).

Workflow and Neural Architecture

The simulation-based inference pipeline is implemented using the `BasicWorkflow` module provided by BayesFlow. This module bundles the main components of the pipeline into a single object: the simulator, the adapter, and the neural networks used for summarization and posterior inference. Within this workflow, simulated observations are first processed by a summary network that extracts a compact representation of the time-series data. This representation is then used by a conditional normalizing flow to model the posterior distribution over the parameters.

Summary network The summary network extracts a fixed-dimensional representation from the observed data. Since the observations consist of time-series signals (current, voltage, and time indices), a dedicated architecture designed for time-series data is employed. The time-series backbone is implemented using the `TimeSeriesNetwork` module provided by BayesFlow. It consists of several one-dimensional convolutional layers for local feature extraction followed by a gated recurrent unit (GRU) that captures longer-range temporal dependencies. The resulting representation is mapped to a fixed-dimensional summary vector. In models that incorporate additional contextual information, such as sequential inference with prior context, an additional summary backbone is used. The contextual inputs are processed separately and later fused with the time-series representation. For simple contextual inputs (e.g. the mean and concentration parameters of a Beta prior), a small multilayer perceptron (MLP) is used to produce a low-dimensional embedding. For the sequential model, where the context consists of a set of posterior samples from the previous time window, the order of the samples is not meaningful. The context is therefore processed using a `SetTransformer`, which provides a permutation-invariant embedding of the sample set. The outputs of the different summary backbones are combined using a `FusionNetwork`, implemented as a small multilayer perceptron (MLP), which concatenates the summary vectors and maps them to the final latent representation used for inference.

Inference network Posterior estimation is implemented using the `CouplingFlow` module provided by BayesFlow. The flow uses a multivariate normal base distribution and a sequence of spline-based coupling transformations parameterized by small multilayer perceptrons (MLPs). The transformations are conditioned on the summary representation produced by the summary network. Training is performed using BayesFlow's `fit_online` routine, which generates simulated data on-the-fly and optimizes the conditional negative log-likelihood of the flow.

Resources To get started with BayesFlow, the tutorials provided on the official BayesFlow website are recommended.[1] For a broader introduction to simulation-based inference (SBI) and its evaluation methodology, the reader is referred to [18], [13] and [12] which

[1] `https://bayesflow.org/main/examples.html`

provide comprehensive discussions of SBI workflows and principled diagnostic strategies such as simulation-based calibration.

Bibliography

[1] A. Gailani, M. Al-Greer, M. Short, and T. Crosbie, "Degradation cost analysis of li-ion batteries in the capacity market with different degradation models," *Electronics*, vol. 9, no. 1, 2020. [Online]. Available: https://www.mdpi.com/2079-9292/9/1/90

[2] Center for Advanced Life Cycle Engineering (CALCE). Battery research data. University of Maryland. [Online]. Available: https://calce.umd.edu/data

[3] M. Aitio, S. G. Marquis, M. Ascencio, and D. A. Howey, "Bayesian parameter estimation applied to the lithium-ion battery single particle model with electrolyte dynamics," *Journal of The Electrochemical Society*, vol. 167, no. 11, p. 110538, 2020.

[4] S. A. Sisson, Y. Fan, and M. A. Beaumont, *Handbook of approximate Bayesian computation*. Boca Raton, Florida ;: CRC Press, 2019.

[5] L. F. Price, C. C. Drovandi, A. Lee, and D. J. Nott, "Bayesian synthetic likelihood," *Journal of Computational and Graphical Statistics*, vol. 27, no. 1, pp. 1–11, 2018. [Online]. Available: https://doi.org/10.1080/10618600.2017.1302882

[6] L. N. Kumari and P. Wang, "Efficient stochastic parametric estimation for lithium-ion battery performance degradation tracking and prognosis," *Journal of Manufacturing Systems*, vol. 75, pp. 270–277, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0278612524000669

[7] S. T. Radev, M. Schmitt, L. Schumacher, L. Elsemüller, V. Pratz, Y. Schälte, U. Köthe, and P.-C. Bürkner, "Bayesflow: Amortized bayesian workflows with neural networks," 2023. [Online]. Available: https://arxiv.org/abs/2306.16015

[8] L. Fontanella and L. Ippoliti, "17 - karhunen–loéve expansion of temporal and spatio-temporal processes," in *Time Series Analysis: Methods and Applications*, ser. Handbook of Statistics, T. Subba Rao, S. Subba Rao, and C. Rao, Eds. Elsevier, 2012, vol. 30, pp. 497–520. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B978044453858100017X

[9] L. Dinh, J. Sohl-Dickstein, and S. Bengio, "Density estimation using real nvp," 2017. [Online]. Available: https://arxiv.org/abs/1605.08803

[10] C. Durkan, A. Bekasov, I. Murray, and G. Papamakarios, "Neural spline flows," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/7ac71d433f282034e088473244df8c02-Paper.pdf

[11] Y. Yang, S. Rissanen, P. E. Chang, N. Loka, D. Huang, A. Solin, M. Heinonen, and L. Acerbi, "Priorguide: Test-time prior adaptation for simulation-based inference," 2025. [Online]. Available: https://arxiv.org/abs/2510.13763

[12] S. Talts, M. Betancourt, D. Simpson, A. Vehtari, and A. Gelman, "Validating bayesian inference algorithms with simulation-based calibration," 2020. [Online]. Available: https://arxiv.org/abs/1804.06788

[13] J.-M. Lueckmann, J. Boelts, D. Greenberg, P. Goncalves, and J. Macke, "Benchmarking simulation-based inference," in *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, A. Banerjee and K. Fukumizu, Eds., vol. 130. PMLR, 13–15 Apr 2021, pp. 343–351. [Online]. Available: https://proceedings.mlr.press/v130/lueckmann21a.html

[14] M. Modrák, S. Kim, A. H. Moon, and T. Säilynoja, "Sbc rank visualizations," 2025. [Online]. Available: https://hyunjimoon.github.io/SBC/articles/rank_visualizations.html

[15] A. Gelman, J. B. Carlin, H. S. Stern, and D. B. Rubin, *Bayesian Data Analysis*, 2nd ed. Chapman and Hall/CRC, 2004. [Online]. Available: https://toc.library.ethz.ch/objects/pdf_uzh50/4/978-1-58488-388-3_005529652.pdf

[16] L. Tiberi and M. Baur, "Dynamic multi-scale dual kalman filtering for state of charge and capacity estimation on lithium titanium oxide cells for traction applications," in *2023 International Conference on Clean Electrical Power (ICCEP)*, 2023, pp. 344–354.

[17] Y. Schälte, E. Klinger, E. Alamoudi, and J. Hasenauer, "pyabc: Efficient and robust easy-to-use approximate bayesian computation," *Journal of Open Source Software*, vol. 7, no. 74, p. 4304, 2022. [Online]. Available: https://doi.org/10.21105/joss.04304

[18] M. Deistler, J. Boelts, P. Steinbach, G. Moss, T. Moreau, M. Gloeckler, P. L. C. Rodrigues, J. Linhart, J. K. Lappalainen, B. K. Miller, P. J. Gonçalves, J.-M. Lueckmann, C. Schröder, and J. H. Macke, "Simulation-based inference: A practical guide," 2025. [Online]. Available: https://arxiv.org/abs/2508.12939